

IF2211 Strategi Algoritma

Tugas Kecil 3

Ice Sliding Puzzle Solver



Dipersiapkan oleh:

13524046 - Farrell Limjaya

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2026**

I. Algoritma Pathfinding

1. Uniform Cost Search (UCS)

Uniform Cost Search adalah algoritma pathfinding yang memilih state berdasarkan total cost terkecil dari posisi awal. Pada permainan Ice Sliding Puzzle, cost tidak dihitung dari banyaknya gerakan, tetapi dari jumlah cost seluruh tile yang dilewati ketika aktor melakukan sliding. Oleh karena itu, UCS cocok digunakan karena algoritma ini selalu memprioritaskan jalur dengan total biaya paling kecil.

Pada implementasi program, satu state terdiri dari posisi aktor pada papan dan digit berikutnya yang harus dilewati. Digit berikutnya disimpan karena permainan mewajibkan aktor melewati angka secara berurutan. Dengan demikian, dua state dengan posisi aktor yang sama tetap dapat dianggap berbeda apabila digit yang sudah dilewati berbeda.

UCS bekerja dengan menggunakan priority queue. State awal dimasukkan ke dalam priority queue dengan cost 0. Setelah itu, program akan selalu mengambil state dengan cost terkecil untuk diproses. Dari state tersebut, program mencoba empat kemungkinan arah gerak, yaitu atas, kanan, bawah, dan kiri. Setiap arah akan diproses dengan aturan sliding, yaitu aktor terus bergerak sampai berhenti sebelum rintangan, melewati lava, keluar papan, atau mencapai titik tujuan yang valid.

Jika hasil sliding valid, program menghitung total cost baru. Jika cost baru tersebut lebih kecil daripada cost terbaik yang pernah tercatat untuk state tersebut, maka program memperbarui data state, menyimpan parent, menyimpan arah gerakan, lalu memasukkan state baru ke priority queue. Proses ini terus dilakukan sampai state tujuan ditemukan atau priority queue kosong.

Pseudocode Uniform Cost Search:

PROCEDURE Uniform_Cost_Search(board)

 buat priority queue kosong

 buat array dist untuk menyimpan cost minimum setiap state

 buat array parent untuk menyimpan state sebelumnya

 buat array parentAction untuk menyimpan arah gerakan

 FOR setiap state yang mungkin DO

 dist[state] $\leftarrow$ infinity

 parent[state] $\leftarrow$ kosong

 parentAction[state] $\leftarrow$ kosong

 ENDFOR

startState $\leftarrow$ state(posisi awal aktor, nextDigit = 0)

dist[startState] $\leftarrow$ 0

masukkan startState ke priority queue dengan prioritas 0

WHILE priority queue tidak kosong DO

 current $\leftarrow$ ambil state dengan prioritas terkecil dari priority queue

 IF current bukan state dengan cost terbaik pada dist THEN

 lanjut ke iterasi berikutnya

 ENDIF

 IF current berada pada titik tujuan DAN semua digit sudah dilewati THEN

 finalState $\leftarrow$ current

 solusi ditemukan

 hentikan pencarian

 ENDIF

FOR setiap arah dir dalam {U, R, D, L} DO

 hasilGerak $\leftarrow$ lakukan sliding dari current ke arah dir

 IF hasilGerak valid THEN

 nextState $\leftarrow$ state hasil sliding

moveCost $\leftarrow$ total cost tile yang dilewati saat sliding

newCost $\leftarrow$ dist[current] + moveCost

IF newCost < dist[nextState] THEN

dist[nextState] $\leftarrow$ newCost

parent[nextState] $\leftarrow$ current

parentAction[nextState] $\leftarrow$ dir

masukkan nextState ke priority queue

dengan prioritas newCost

ENDIF

ENDIF

ENDFOR

ENDWHILE

IF solusi ditemukan THEN

bentuk kembali urutan gerakan dari finalState

menggunakan parent dan parentAction

ELSE

tampilkan bahwa solusi tidak ditemukan

ENDIF

END PROCEDURE

Pada pseudocode tersebut, dist digunakan untuk menyimpan cost terbaik menuju setiap state. Jika suatu state ditemukan kembali dengan cost yang lebih besar, maka state tersebut tidak perlu diproses karena sudah ada jalur yang lebih murah. Sebaliknya, jika ditemukan jalur dengan cost lebih kecil, maka data state diperbarui.

Bagian parent dan parentAction digunakan untuk membentuk kembali solusi setelah tujuan ditemukan. Program tidak menyimpan seluruh jalur pada setiap state, melainkan hanya menyimpan state sebelumnya dan arah gerakan yang digunakan. Setelah finalState ditemukan, program menelusuri kembali parent dari finalState sampai startState untuk mendapatkan urutan gerakan solusi.

Karena UCS selalu memproses state dengan total cost terkecil, algoritma ini dapat menghasilkan solusi dengan cost minimum selama seluruh cost traversal tidak negatif. Kekurangannya, UCS dapat meninjau banyak state karena tidak menggunakan informasi perkiraan arah menuju tujuan.

2. Greedy Best First Search (GBFS)

Greedy Best First Search adalah algoritma pathfinding yang memilih state berdasarkan nilai heuristic. Berbeda dengan UCS yang memilih state berdasarkan total cost dari posisi awal, GBFS memilih state yang diperkirakan paling dekat dengan target. Target dalam program ini dapat berupa digit berikutnya yang harus dilewati atau titik tujuan apabila semua digit sudah dilewati.

Pada implementasi program, GBFS tetap menggunakan priority queue seperti UCS. Namun, prioritas state tidak ditentukan oleh total cost dari awal, melainkan oleh nilai heuristic. Semakin kecil nilai heuristic suatu state, semakin tinggi prioritas state tersebut untuk diproses. Dengan cara ini, GBFS cenderung bergerak lebih cepat ke arah target.

Langkah awal GBFS adalah memasukkan state awal ke priority queue dengan prioritas berupa nilai heuristic dari state awal. Setelah itu, program mengambil state dengan heuristic terkecil. Dari state tersebut, program mencoba empat kemungkinan arah gerak, yaitu U, R, D, dan L. Setiap gerakan diproses menggunakan aturan sliding yang sama seperti UCS.

Jika gerakan valid, program menghasilkan state baru dan menghitung cost aktual yang dikeluarkan. Cost aktual tetap disimpan agar program dapat mengetahui total biaya solusi yang ditemukan. Namun, untuk menentukan state mana yang diproses lebih dahulu, GBFS hanya menggunakan nilai heuristic. State baru dimasukkan ke priority queue dengan prioritas heuristic-nya.

Pseudocode Greedy Best First Search:

PROCEDURE Greedy_Best_First_Search(board, heuristic)

 buat priority queue kosong

 buat array dist untuk menyimpan cost terbaik setiap state

 buat array parent untuk menyimpan state sebelumnya

 buat array parentAction untuk menyimpan arah gerakan

 FOR setiap state yang mungkin DO

 dist[state] $\leftarrow$ infinity

parent[state] ← kosong

parentAction[state] ← kosong

ENDFOR

startState ← state(posisi awal aktor, nextDigit = 0)

dist[startState] ← 0

hStart ← heuristic(startState)

masukkan startState ke priority queue dengan prioritas hStart

WHILE priority queue tidak kosong DO

current ← ambil state dengan nilai heuristic terkecil

dari priority queue

IF current bukan state dengan cost terbaik pada dist THEN

lanjut ke iterasi berikutnya

ENDIF

IF current berada pada titik tujuan DAN semua digit sudah dilewati THEN

finalState ← current

solusi ditemukan

hentikan pencarian

ENDIF

FOR setiap arah dir dalam {U, R, D, L} DO

hasilGerak $\leftarrow$ lakukan sliding dari current ke arah dir

IF hasilGerak valid THEN

 nextState $\leftarrow$ state hasil sliding

 moveCost $\leftarrow$ total cost tile yang dilewati saat sliding

 newCost $\leftarrow$ dist[current] + moveCost

IF newCost < dist[nextState] THEN

 dist[nextState] $\leftarrow$ newCost

 parent[nextState] $\leftarrow$ current

 parentAction[nextState] $\leftarrow$ dir

 hNext $\leftarrow$ heuristic(nextState)

 masukkan nextState ke priority queue

 dengan prioritas hNext

ENDIF

ENDIF

ENDFOR

ENDWHILE

IF solusi ditemukan THEN

 bentuk kembali urutan gerakan dari finalState

menggunakan parent dan parentAction

ELSE

tampilkan bahwa solusi tidak ditemukan

ENDIF

END PROCEDURE

Pada pseudocode GBFS, perbedaan utama dengan UCS terdapat pada nilai prioritas yang dimasukkan ke priority queue. Pada UCS, prioritas menggunakan newCost. Pada GBFS, prioritas menggunakan heuristic dari state baru. Artinya, GBFS lebih memilih state yang terlihat dekat dengan target, bukan state yang memiliki total cost terkecil dari posisi awal.

Dalam program ini, heuristic digunakan untuk memperkirakan jarak menuju target berikutnya. Jika masih ada digit yang belum dilewati, target berikutnya adalah digit tersebut. Jika semua digit sudah dilewati, target berikutnya adalah titik tujuan. Dengan cara ini, GBFS dapat mengarahkan pencarian menuju bagian papan yang dianggap lebih menjanjikan.

Keunggulan GBFS adalah pencariannya dapat lebih cepat dalam beberapa kasus karena tidak menelusuri state berdasarkan cost secara menyeluruh seperti UCS. Namun, GBFS tidak menjamin solusi dengan cost minimum. Hal ini karena state yang terlihat dekat berdasarkan heuristic belum tentu memiliki jalur dengan cost paling murah.

3. A* Search

A* adalah algoritma pathfinding yang menggabungkan pendekatan Uniform Cost Search dan Greedy Best First Search. Jika UCS memilih state berdasarkan total cost dari posisi awal, dan GBFS memilih state berdasarkan heuristic saja, maka A* memilih state berdasarkan gabungan keduanya. Dengan demikian, A* tidak hanya memperhatikan biaya yang sudah dikeluarkan, tetapi juga memperkirakan jarak menuju target.

Pada program ini, A* menggunakan nilai $g(n)$ dan $h(n)$. Nilai $g(n)$ adalah total cost dari posisi awal sampai state saat ini. Cost tersebut diperoleh dari jumlah biaya seluruh tile yang dilewati selama aktor melakukan sliding. Sementara itu, $h(n)$ adalah nilai heuristic yang memperkirakan jarak dari state saat ini menuju target berikutnya. Target berikutnya dapat berupa digit yang harus dilewati atau titik tujuan apabila semua digit sudah dilewati.

A* menggunakan priority queue untuk memilih state yang akan diproses. Setiap state diberi prioritas berdasarkan nilai $f(n)$, yaitu penjumlahan antara $g(n)$ dan $h(n)$. State dengan nilai $f(n)$ paling kecil akan diproses terlebih dahulu. Dengan cara ini, A* dapat mencari jalur yang tetap mempertimbangkan cost aktual, tetapi lebih terarah karena dibantu oleh heuristic.

Langkah awal A* adalah memasukkan state awal ke priority queue. State awal memiliki cost $g(n) = 0$. Kemudian, program menghitung heuristic dari state awal dan menjadikannya bagian dari prioritas. Setelah itu, program mengambil state dengan nilai $f(n)$ terkecil. Jika state tersebut sudah berada pada titik tujuan dan semua digit telah dilewati, maka pencarian dihentikan.

Jika state belum merupakan tujuan, program mencoba empat kemungkinan arah gerak, yaitu U, R, D, dan L. Setiap gerakan diproses menggunakan aturan sliding. Jika gerakan valid, program menghasilkan state baru, menghitung cost tambahan gerakan, lalu menghitung total cost baru. Jika total cost baru lebih kecil daripada cost terbaik yang pernah ditemukan untuk state tersebut, maka state diperbarui dan dimasukkan kembali ke priority queue dengan prioritas baru.

Pseudocode A* Search:

PROCEDURE A_Star_Search(board, heuristic)

 buat priority queue kosong

 buat array dist untuk menyimpan cost terbaik setiap state

 buat array parent untuk menyimpan state sebelumnya

 buat array parentAction untuk menyimpan arah gerakan

 FOR setiap state yang mungkin DO

 dist[state] $\leftarrow$ infinity

 parent[state] $\leftarrow$ kosong

 parentAction[state] $\leftarrow$ kosong

 ENDFOR

 startState $\leftarrow$ state(posisi awal aktor, nextDigit = 0)

 dist[startState] $\leftarrow$ 0

 hStart $\leftarrow$ heuristic(startState)

 fStart $\leftarrow$ dist[startState] + hStart

 masukkan startState ke priority queue dengan prioritas fStart

 WHILE priority queue tidak kosong DO

current $\leftarrow$ ambil state dengan nilai f paling kecil
dari priority queue

IF current bukan state dengan cost terbaik pada dist THEN
 lanjut ke iterasi berikutnya
ENDIF

IF current berada pada titik tujuan DAN semua digit sudah dilewati THEN
 finalState $\leftarrow$ current
 solusi ditemukan
 hentikan pencarian
ENDIF

FOR setiap arah dir dalam {U, R, D, L} DO

 hasilGerak $\leftarrow$ lakukan sliding dari current ke arah dir

 IF hasilGerak valid THEN

 nextState $\leftarrow$ state hasil sliding

 moveCost $\leftarrow$ total cost tile yang dilewati saat sliding

 newCost $\leftarrow$ dist[current] + moveCost

 IF newCost < dist[nextState] THEN

 dist[nextState] $\leftarrow$ newCost

 parent[nextState] $\leftarrow$ current

 parentAction[nextState] $\leftarrow$ dir

```

        hNext ← heuristic(nextState)

        fNext ← newCost + hNext

        masukkan nextState ke priority queue

        dengan prioritas fNext

    ENDIF

ENDIF

ENDFOR

ENDWHILE

IF solusi ditemukan THEN

    bentuk kembali urutan gerakan dari finalState

    menggunakan parent dan parentAction

ELSE

    tampilkan bahwa solusi tidak ditemukan

ENDIF

END PROCEDURE

```

Pada pseudocode A*, bagian paling penting adalah perhitungan fNext. Nilai fNext diperoleh dari $\text{newCost} + \text{hNext}$. newCost merupakan total cost aktual dari posisi awal sampai nextState, sedangkan hNext merupakan estimasi jarak dari nextState menuju target berikutnya. Karena menggunakan dua komponen tersebut, A* dapat menyeimbangkan antara jalur yang murah dan jalur yang terlihat dekat dengan tujuan.

Perbedaan A* dengan UCS terletak pada penggunaan heuristic. UCS hanya menggunakan cost aktual, sehingga pencarian dapat melebar ke banyak state. A* tetap menggunakan cost aktual, tetapi menambahkan heuristic agar pencarian lebih terarah. Sementara itu, perbedaan A* dengan GBFS adalah A* tidak hanya melihat heuristic, tetapi juga tetap memperhitungkan total cost dari awal.

Dalam konteks Ice Sliding Puzzle, A* cocok digunakan karena setiap gerakan memiliki cost yang berbeda-beda. Jika hanya memakai heuristic seperti GBFS, solusi yang ditemukan bisa saja cepat tetapi tidak murah. Sebaliknya, jika hanya memakai UCS, solusi yang ditemukan optimal tetapi pencarian dapat memeriksa lebih banyak state. A* berada di antara keduanya karena mempertimbangkan cost aktual dan estimasi menuju target secara bersamaan.

Secara umum, A* dapat memberikan hasil yang lebih seimbang dibanding UCS dan GBFS. Algoritma ini dapat lebih cepat daripada UCS karena dibantu heuristic, dan dapat menghasilkan solusi yang lebih baik daripada GBFS karena tetap memperhatikan total cost perjalanan dari posisi awal.

II. Analisis Algoritma Pathfinding

Algoritma Uniform Cost Search, Greedy Best First Search, dan A* memiliki cara kerja yang mirip dalam membangkitkan state, tetapi berbeda dalam menentukan state mana yang diproses terlebih dahulu.

Dalam program ini, satu state n merepresentasikan kondisi permainan yang terdiri dari posisi aktor pada papan dan digit berikutnya yang harus dilewati. State tidak hanya bergantung pada posisi baris dan kolom aktor, karena progress angka juga memengaruhi validitas solusi. Misalnya, aktor yang berada pada posisi yang sama dapat dianggap berada pada state berbeda apabila angka yang sudah dilewati berbeda.

1. Definisi $g(n)$, $h(n)$, dan $f(n)$

Pada algoritma pathfinding, $g(n)$, $h(n)$, dan $f(n)$ digunakan untuk menentukan prioritas suatu state dalam proses pencarian.

$g(n)$ adalah total cost aktual dari state awal sampai state n . Pada program ini, $g(n)$ dihitung dari penjumlahan seluruh cost movement yang telah dilakukan. Cost movement sendiri diperoleh dari jumlah cost seluruh tile yang dilewati selama satu gerakan sliding. Tile awal tidak dihitung, tetapi tile yang dilewati dan tile tempat aktor berhenti tetap dihitung.

$h(n)$ adalah nilai heuristic atau estimasi jarak dari state n menuju target berikutnya. Target berikutnya dapat berupa digit yang harus dilewati sesuai urutan, atau titik tujuan O apabila seluruh digit sudah dilewati. Nilai $h(n)$ digunakan pada GBFS dan A* untuk membantu mengarahkan pencarian agar tidak memeriksa state secara terlalu luas.

$f(n)$ adalah nilai prioritas yang digunakan untuk menentukan state mana yang akan diproses terlebih dahulu dari priority queue. Setiap algoritma memiliki definisi $f(n)$ yang berbeda.

Pada Uniform Cost Search, fungsi prioritasnya adalah:

$$f(n) = g(n)$$

UCS hanya memperhatikan total cost aktual dari posisi awal sampai state saat ini. Karena itu, state dengan $g(n)$ paling kecil akan diproses terlebih dahulu. UCS tidak menggunakan heuristic.

Pada Greedy Best First Search, fungsi prioritasnya adalah:

$$f(n) = h(n)$$

GBFS hanya memperhatikan estimasi jarak menuju target. State yang terlihat paling dekat dengan target akan diproses terlebih dahulu, tanpa menjadikan total cost dari awal sebagai prioritas utama.

Pada A*, fungsi prioritasnya adalah:

$$f(n) = g(n) + h(n)$$

A* menggabungkan cost aktual dan estimasi jarak ke target. Dengan cara ini, A* tidak hanya memilih state yang murah dari awal, tetapi juga mempertimbangkan apakah state tersebut menjanjikan untuk mencapai target.

2. Admissibility Heuristic pada A*

Heuristic dikatakan admissible apabila nilai estimasi $h(n)$ tidak pernah lebih besar daripada cost sebenarnya dari state n menuju tujuan. Dengan kata lain, heuristic boleh meremehkan jarak atau cost menuju tujuan, tetapi tidak boleh melebihinya. Jika heuristic admissible, maka A* dapat menghasilkan solusi optimal, selama cost pada graf tidak bernilai negatif.

Pada program ini terdapat tiga heuristic, yaitu H1, H2, dan H3.

H1 menggunakan jarak Manhattan dari posisi aktor saat ini menuju target berikutnya. Target berikutnya adalah digit yang harus dilewati, atau titik tujuan jika seluruh digit sudah dilewati. Jarak Manhattan digunakan karena pergerakan pada papan hanya dapat terjadi secara horizontal dan vertikal. H1 dapat dianggap admissible selama cost minimum tile yang dapat dilewati bernilai paling sedikit 1, karena jumlah tile yang harus dilalui secara nyata tidak mungkin lebih kecil daripada jarak Manhattan ke target.

H2 menggunakan jarak Manhattan seperti H1, tetapi dikalikan dengan cost minimum dari tile yang dapat dilewati. Dengan demikian, H2 memberikan estimasi berbasis cost, bukan hanya estimasi jumlah langkah. H2 juga admissible selama cost minimum yang digunakan benar-benar merupakan batas bawah dari semua cost tile yang dapat dilewati. Karena setiap jalur valid pasti melewati sejumlah tile, dan setiap tile memiliki cost minimal sebesar cost minimum tersebut, maka hasil perkalian Manhattan distance dengan cost minimum tidak akan melebihi cost sebenarnya.

H3 menggunakan estimasi jarak dari posisi saat ini menuju semua digit yang tersisa secara berurutan, lalu menuju titik tujuan O. Total jarak tersebut kemudian dikalikan dengan cost minimum tile yang dapat dilewati. H3 lebih informatif dibanding H1 dan H2 karena memperhitungkan bahwa aktor tidak hanya perlu mencapai target berikutnya, tetapi juga harus menyelesaikan seluruh urutan digit sampai akhirnya mencapai tujuan. H3 dapat dianggap admissible karena urutan digit memang wajib dilalui. Setiap solusi valid pasti harus menempuh jalur dari posisi sekarang menuju digit berikutnya, lalu ke digit selanjutnya, sampai akhirnya menuju O. Dengan mengalikan jarak minimum tersebut dengan cost tile minimum, estimasi yang diperoleh tetap menjadi batas bawah dari cost sebenarnya.

Namun, admissibility tersebut bergantung pada asumsi bahwa seluruh cost tile yang dapat dilewati bernilai positif dan cost minimum yang digunakan benar-benar merupakan nilai cost terkecil dari tile valid. Jika terdapat tile valid dengan cost 0, maka heuristic berbasis jarak dapat berpotensi melebihi cost sebenarnya. Pada data uji yang digunakan, cost traversal pada tile yang dapat dilewati bernilai positif, sehingga heuristic H1, H2, dan H3 dapat digunakan sebagai estimasi yang admissible untuk A^* .

Di antara ketiga heuristic tersebut, H1 adalah yang paling sederhana karena hanya melihat target berikutnya. H2 lebih baik karena mulai mempertimbangkan cost minimum. H3 adalah heuristic yang paling informatif karena memperkirakan perjalanan sampai seluruh digit selesai dan tujuan dicapai. Semakin informatif heuristic, semakin terarah pencarian yang dilakukan A^* .

3. Perbandingan Teoritis Algoritma

Secara teoritis, Uniform Cost Search, Greedy Best First Search, dan A^* memiliki karakteristik yang berbeda dalam menyelesaikan Ice Sliding Puzzle.

Uniform Cost Search memiliki keunggulan dalam hal optimalitas. Karena UCS selalu memilih state dengan total cost terkecil dari posisi awal, solusi pertama yang ditemukan oleh UCS merupakan solusi dengan cost minimum, selama semua cost tidak negatif. Dalam permasalahan ini, UCS cocok digunakan sebagai pembanding karena dapat memberikan hasil optimal. Kekurangan UCS adalah jumlah state yang diperiksa dapat sangat banyak, terutama pada papan besar atau papan dengan banyak kemungkinan gerakan, karena UCS tidak memiliki informasi arah menuju tujuan.

Greedy Best First Search memiliki keunggulan dalam kecepatan pencarian pada beberapa kasus. GBFS memilih state berdasarkan heuristic, sehingga pencarian lebih diarahkan menuju target berikutnya. Jika heuristic cukup baik, GBFS dapat menemukan solusi dengan jumlah iterasi yang lebih sedikit dibanding UCS. Namun, GBFS tidak menjamin solusi optimal. Hal ini karena GBFS tidak memprioritaskan total cost dari awal, sehingga jalur yang terlihat dekat berdasarkan heuristic belum tentu merupakan jalur dengan cost paling kecil.

A^* berada di antara UCS dan GBFS. A^* menggunakan $g(n)$ seperti UCS untuk memperhatikan cost aktual, dan menggunakan $h(n)$ seperti GBFS untuk mengarahkan pencarian. Dengan heuristic yang admissible, A^* dapat menghasilkan solusi optimal seperti UCS, tetapi biasanya dengan jumlah state yang lebih sedikit karena pencariannya lebih terarah. Dalam kasus Ice Sliding Puzzle, A^* cocok digunakan karena setiap gerakan memiliki cost berbeda dan posisi target dapat diperkirakan menggunakan heuristic.

Jika dibandingkan berdasarkan optimalitas, UCS dan A^* dapat menghasilkan solusi optimal, sedangkan GBFS tidak menjamin optimal. Jika dibandingkan berdasarkan jumlah state yang diperiksa, UCS cenderung memeriksa state paling banyak karena tidak memakai heuristic. GBFS dapat memeriksa state lebih sedikit, tetapi kualitas solusinya tidak selalu terbaik. A^* umumnya lebih seimbang karena tetap mempertimbangkan cost dan heuristic.

Jika dibandingkan berdasarkan kebutuhan memori, ketiga algoritma membutuhkan memori untuk menyimpan state, cost terbaik, parent, dan priority queue. Pada implementasi ini, jumlah state maksimum bergantung pada ukuran papan dan jumlah digit yang harus dilewati.

Jika ukuran papan adalah $N \times M$ dan jumlah digit yang harus dilewati adalah K , maka jumlah state maksimum adalah sekitar $N \times M \times (K + 1)$. Setiap state dapat menghasilkan hingga empat kemungkinan gerakan, yaitu atas, kanan, bawah, dan kiri.

Dengan demikian, secara teoritis UCS paling aman untuk mencari cost minimum tetapi dapat lebih lambat, GBFS dapat lebih cepat tetapi tidak optimal, sedangkan A* menjadi pilihan yang paling seimbang karena menggabungkan cost aktual dan heuristic dalam proses pencarian.

III. Source Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <limits.h>
#include <time.h>
#include <sys/stat.h>

#ifdef _WIN32
#include <conio.h>
#include <windows.h>
#include <direct.h>
#else
#include <termios.h>
#include <unistd.h>
#include <sys/select.h>
#endif

#define INF (LLONG_MAX / 4)
#define MAX_PATH_LEN 256

typedef enum {
    ALG_UCS,
    ALG_GBFS,
    ALG_ASTAR
} Algorithm;

typedef enum {
    HEUR_H1,
    HEUR_H2,
    HEUR_H3
} HeuristicType;
```

```
typedef struct {
    int n, m;
    char **grid;
    long long **cost;
    int startR, startC;
    int goalR, goalC;
    int maxDigit;
    int digitExists[10];
    int digitR[10];
    int digitC[10];
    long long minPositiveCost;
} Board;
```

```
typedef struct {
    int r, c;
    int nextDigit;
    long long moveCost;
} MoveResult;
```

```
typedef struct {
    int key;
    long long priority;
    long long g;
    long long order;
} PQItem;
```

```
typedef struct {
    PQItem *data;
    int size;
    int capacity;
    long long pushOrder;
} PriorityQueue;
```

```
typedef struct {
    int *data;
    int size;
    int capacity;
} IntVector;
```

```
typedef struct {
    int found;
```

```

    int finalKey;
    long long finalCost;
    long long expandedCount;
    double execMs;
    int *parent;
    char *parentAction;
    long long *dist;
    IntVector expandedTrace;
} SearchResult;

typedef struct {
    int *keys;
    char *actions;
    int moveCount;
} SolutionPath;

static const int DR[4] = {-1, 0, 1, 0};
static const int DC[4] = {0, 1, 0, -1};
static const char ACT[4] = {'U', 'R', 'D', 'L'};

static void clear_screen(void) {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

static int equals_ignore_case(const char *a, const char *b) {
    while (*a && *b) {
        if (tolower((unsigned char)*a) != tolower((unsigned char)*b)) return 0;
        a++;
        b++;
    }
    return *a == '\0' && *b == '\0';
}

static int yes_answer(const char *s) {
    return equals_ignore_case(s, "ya") ||
        equals_ignore_case(s, "y") ||
        equals_ignore_case(s, "yes");
}

```

```

static int has_path_separator(const char *s) {
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '/' || s[i] == '\\') return 1;
    }
    return 0;
}

#ifdef _WIN32
static int directory_exists(const char *path) {
    DWORD attr = GetFileAttributesA(path);
    return attr != INVALID_FILE_ATTRIBUTES && (attr & FILE_ATTRIBUTE_DIRECTORY);
}

static int file_exists(const char *path) {
    DWORD attr = GetFileAttributesA(path);
    return attr != INVALID_FILE_ATTRIBUTES && !(attr &
FILE_ATTRIBUTE_DIRECTORY);
}

static int make_directory_if_missing(const char *path) {
    if (directory_exists(path)) return 1;
    return _mkdir(path) == 0 || directory_exists(path);
}
#else
static int directory_exists(const char *path) {
    struct stat st;
    return stat(path, &st) == 0 && S_ISDIR(st.st_mode);
}

static int file_exists(const char *path) {
    struct stat st;
    return stat(path, &st) == 0 && !S_ISDIR(st.st_mode);
}

static int make_directory_if_missing(const char *path) {
    if (directory_exists(path)) return 1;
    return mkdir(path, 0755) == 0 || directory_exists(path);
}
#endif

static FILE *open_input_file(const char *filename, char *resolvedPath, size_t resolvedSize) {

```

```

FILE *fp = fopen(filename, "r");
if (fp) {
    snprintf(resolvedPath, resolvedSize, "%s", filename);
    return fp;
}

if (!has_path_separator(filename)) {
    char candidate[MAX_PATH_LEN * 2];
    snprintf(candidate, sizeof(candidate), "src/%s", filename);
    fp = fopen(candidate, "r");
    if (fp) {
        snprintf(resolvedPath, resolvedSize, "%s", candidate);
        return fp;
    }

    snprintf(candidate, sizeof(candidate), "./src/%s", filename);
    fp = fopen(candidate, "r");
    if (fp) {
        snprintf(resolvedPath, resolvedSize, "%s", candidate);
        return fp;
    }
}

resolvedPath[0] = '\0';
return NULL;
}

static void resolve_output_path(const char *inputName, char *resolvedPath, size_t resolvedSize)
{
    if (has_path_separator(inputName)) {
        snprintf(resolvedPath, resolvedSize, "%s", inputName);
        return;
    }

    if (file_exists("main.c") && !directory_exists("src")) {
        make_directory_if_missing("../test");
        snprintf(resolvedPath, resolvedSize, "../test/%s", inputName);
    } else {
        make_directory_if_missing("test");
        snprintf(resolvedPath, resolvedSize, "test/%s", inputName);
    }
}

```

```

static int parse_algorithm(const char *s, Algorithm *alg) {
    if (equals_ignore_case(s, "UCS")) {
        *alg = ALG_UCS;
        return 1;
    }
    if (equals_ignore_case(s, "GBFS")) {
        *alg = ALG_GBFS;
        return 1;
    }
    if (equals_ignore_case(s, "A*") ||
        equals_ignore_case(s, "ASTAR") ||
        equals_ignore_case(s, "A_STAR")) {
        *alg = ALG_ASTAR;
        return 1;
    }
    return 0;
}

```

```

static int parse_heuristic(const char *s, HeuristicType *h) {
    if (equals_ignore_case(s, "H1")) {
        *h = HEUR_H1;
        return 1;
    }
    if (equals_ignore_case(s, "H2")) {
        *h = HEUR_H2;
        return 1;
    }
    if (equals_ignore_case(s, "H3")) {
        *h = HEUR_H3;
        return 1;
    }
    return 0;
}

```

```

static const char *algorithm_name(Algorithm alg) {
    switch (alg) {
        case ALG_UCS: return "UCS";
        case ALG_GBFS: return "GBFS";
        case ALG_ASTAR: return "A*";
        default: return "UNKNOWN";
    }
}

```

```

static const char *heuristic_name(HeuristicType h) {
    switch (h) {
        case HEUR_H1: return "H1";
        case HEUR_H2: return "H2";
        case HEUR_H3: return "H3";
        default: return "UNKNOWN";
    }
}

static int is_digit_tile(char ch) {
    return ch >= '0' && ch <= '9';
}

static int is_valid_tile(char ch) {
    return ch == '*' || ch == 'X' || ch == 'L' || ch == 'Z' || ch == 'O' || is_digit_tile(ch);
}

static void init_board(Board *b) {
    memset(b, 0, sizeof(Board));
    b->startR = b->startC = -1;
    b->goalR = b->goalC = -1;
    b->maxDigit = -1;
    b->minPositiveCost = INF;
    for (int i = 0; i < 10; i++) {
        b->digitExists[i] = 0;
        b->digitR[i] = b->digitC[i] = -1;
    }
}

static void free_board(Board *b) {
    if (!b) return;

    if (b->grid) {
        for (int i = 0; i < b->n; i++) {
            free(b->grid[i]);
        }
        free(b->grid);
    }

    if (b->cost) {
        for (int i = 0; i < b->n; i++) {

```

```

        free(b->cost[i]);
    }
    free(b->cost);
}

init_board(b);
}

static int allocate_board(Board *b, int n, int m) {
    b->n = n;
    b->m = m;
    b->grid = (char **)malloc((size_t)n * sizeof(char *));
    b->cost = (long long **)malloc((size_t)n * sizeof(long long *));

    if (!b->grid || !b->cost) return 0;

    for (int i = 0; i < n; i++) {
        b->grid[i] = NULL;
        b->cost[i] = NULL;
    }

    for (int i = 0; i < n; i++) {
        b->grid[i] = (char *)malloc((size_t)m + 1);
        b->cost[i] = (long long *)malloc((size_t)m * sizeof(long long));

        if (!b->grid[i] || !b->cost[i]) return 0;
    }

    return 1;
}

static int load_board(const char *filename, Board *b, char *err, size_t errSize) {
    init_board(b);

    char resolvedPath[MAX_PATH_LEN * 2];
    FILE *fp = open_input_file(filename, resolvedPath, sizeof(resolvedPath));

    if (!fp) {
        snprintf(err, errSize, "File tidak dapat dibuka: %s", filename);
        return 0;
    }
}

```

```
int n, m;
if (fscanf(fp, "%d %d", &n, &m) != 2) {
    snprintf(err, errSize, "Baris pertama harus berisi N dan M.");
    fclose(fp);
    return 0;
}

if (n <= 0 || m <= 0) {
    snprintf(err, errSize, "Ukuran papan harus positif.");
    fclose(fp);
    return 0;
}

if (!allocate_board(b, n, m)) {
    snprintf(err, errSize, "Alokasi memori gagal.");
    fclose(fp);
    free_board(b);
    return 0;
}

char line[4096];
int startCount = 0;
int goalCount = 0;

for (int i = 0; i < n; i++) {
    if (fscanf(fp, "%4095s", line) != 1) {
        snprintf(err, errSize, "Baris papan ke-%d tidak ditemukan.", i + 1);
        fclose(fp);
        free_board(b);
        return 0;
    }

    if ((int)strlen(line) != m) {
        snprintf(err, errSize, "Panjang baris papan ke-%d harus %d karakter.", i + 1, m);
        fclose(fp);
        free_board(b);
        return 0;
    }

    for (int j = 0; j < m; j++) {
        char ch = line[j];
```

```

        if (!is_valid_tile(ch)) {
            snprintf(err, errSize, "Karakter tidak valid '%c' pada (%d,%d).", ch, i, j);
            fclose(fp);
            free_board(b);
            return 0;
        }

        b->grid[i][j] = ch;

        if (ch == 'Z') {
            startCount++;
            b->startR = i;
            b->startC = j;
        } else if (ch == 'O') {
            goalCount++;
            b->goalR = i;
            b->goalC = j;
        } else if (is_digit_tile(ch)) {
            int d = ch - '0';

            if (b->digitExists[d]) {
                snprintf(err, errSize, "Digit %d muncul lebih dari satu kali.", d);
                fclose(fp);
                free_board(b);
                return 0;
            }

            b->digitExists[d] = 1;
            b->digitR[d] = i;
            b->digitC[d] = j;

            if (d > b->maxDigit) b->maxDigit = d;
        }
    }

    b->grid[i][m] = '\0';
}

if (startCount != 1) {
    snprintf(err, errSize, "Papan harus memiliki tepat satu Z sebagai posisi awal.");
    fclose(fp);
    free_board(b);
}

```

```

    return 0;
}

if (goalCount != 1) {
    snprintf(err, errSize, "Papan harus memiliki tepat satu O sebagai tujuan.");
    fclose(fp);
    free_board(b);
    return 0;
}

if (b->maxDigit >= 0) {
    for (int d = 0; d <= b->maxDigit; d++) {
        if (!b->digitExists[d]) {
            snprintf(err, errSize, "Digit wajib harus kontigu dari 0. Digit %d tidak ditemukan.", d);
            fclose(fp);
            free_board(b);
            return 0;
        }
    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        long long c;

        if (fscanf(fp, "%lld", &c) != 1) {
            snprintf(err, errSize, "Cost tile pada (%d,%d) tidak ditemukan.", i, j);
            fclose(fp);
            free_board(b);
            return 0;
        }

        if (c < 0) {
            snprintf(err, errSize, "Cost tile tidak boleh negatif pada (%d,%d).", i, j);
            fclose(fp);
            free_board(b);
            return 0;
        }

        b->cost[i][j] = c;

        char tile = b->grid[i][j];
    }
}

```

```

        if (tile != 'X' && tile != 'L' && c > 0 && c < b->minPositiveCost) {
            b->minPositiveCost = c;
        }
    }
}

if (b->minPositiveCost == INF) {
    b->minPositiveCost = 1;
}

fclose(fp);
return 1;
}

static int target_count(const Board *b) {
    return b->maxDigit >= 0 ? b->maxDigit + 1 : 0;
}

static int state_count(const Board *b) {
    int tc = target_count(b);
    return b->n * b->m * (tc + 1);
}

static int encode_key(const Board *b, int r, int c, int nextDigit) {
    return (nextDigit * b->n + r) * b->m + c;
}

static void decode_key(const Board *b, int key, int *r, int *c, int *nextDigit) {
    *c = key % b->m;
    int tmp = key / b->m;
    *r = tmp % b->n;
    *nextDigit = tmp / b->n;
}

static int abs_int(int x) {
    return x < 0 ? -x : x;
}

static long long manhattan(int r1, int c1, int r2, int c2) {
    return (long long)abs_int(r1 - r2) + abs_int(c1 - c2);
}

```

```

static long long heuristic_value(const Board *b, int r, int c, int nextDigit, HeuristicType hType) {
    int tc = target_count(b);
    long long h = 0;

    if (hType == HEUR_H1 || hType == HEUR_H2) {
        int tr, tcCol;

        if (nextDigit < tc) {
            tr = b->digitR[nextDigit];
            tcCol = b->digitC[nextDigit];
        } else {
            tr = b->goalR;
            tcCol = b->goalC;
        }

        h = manhattan(r, c, tr, tcCol);

        if (hType == HEUR_H2) {
            h *= b->minPositiveCost;
        }

        return h;
    }

    int cr = r;
    int cc = c;

    for (int d = nextDigit; d < tc; d++) {
        h += manhattan(cr, cc, b->digitR[d], b->digitC[d]);
        cr = b->digitR[d];
        cc = b->digitC[d];
    }

    h += manhattan(cr, cc, b->goalR, b->goalC);
    h *= b->minPositiveCost;

    return h;
}

static int slide_move(const Board *b, int r, int c, int nextDigit, int dir, MoveResult *res) {
    int curR = r;
    int curC = c;

```

```
long long totalCost = 0;
int moved = 0;
int newNext = nextDigit;
int tc = target_count(b);

while (1) {
    int nr = curR + DR[dir];
    int nc = curC + DC[dir];

    if (nr < 0 || nr >= b->n || nc < 0 || nc >= b->m) {
        return 0;
    }

    char tile = b->grid[nr][nc];

    if (tile == 'X') {
        if (!moved) return 0;

        res->r = curR;
        res->c = curC;
        res->nextDigit = newNext;
        res->moveCost = totalCost;
        return 1;
    }

    if (tile == 'L') {
        return 0;
    }

    moved = 1;
    totalCost += b->cost[nr][nc];

    if (is_digit_tile(tile)) {
        int d = tile - '0';

        if (d < newNext) {
        } else if (d == newNext && newNext < tc) {
            newNext++;
        } else {
            return 0;
        }
    }
}
```

```

        curR = nr;
        curC = nc;

        if (tile == 'O' && newNext == tc) {
            res->r = curR;
            res->c = curC;
            res->nextDigit = newNext;
            res->moveCost = totalCost;
            return 1;
        }
    }
}

static void pq_init(PriorityQueue *pq) {
    pq->size = 0;
    pq->capacity = 64;
    pq->pushOrder = 0;
    pq->data = (PQItem *)malloc((size_t)pq->capacity * sizeof(PQItem));
}

static void pq_free(PriorityQueue *pq) {
    free(pq->data);
    pq->data = NULL;
    pq->size = 0;
    pq->capacity = 0;
    pq->pushOrder = 0;
}

static int pq_less(PQItem a, PQItem b) {
    if (a.priority != b.priority) return a.priority < b.priority;
    if (a.g != b.g) return a.g < b.g;
    return a.order < b.order;
}

static void pq_swap(PQItem *a, PQItem *b) {
    PQItem t = *a;
    *a = *b;
    *b = t;
}

static int pq_push(PriorityQueue *pq, int key, long long priority, long long g) {

```

```

if (pq->size >= pq->capacity) {
    int newCap = pq->capacity * 2;
    PQItem *newData = (PQItem *)realloc(pq->data, (size_t)newCap * sizeof(PQItem));

    if (!newData) return 0;

    pq->data = newData;
    pq->capacity = newCap;
}

int idx = pq->size++;

pq->data[idx].key = key;
pq->data[idx].priority = priority;
pq->data[idx].g = g;
pq->data[idx].order = pq->pushOrder++;

while (idx > 0) {
    int parent = (idx - 1) / 2;

    if (!pq_less(pq->data[idx], pq->data[parent])) break;

    pq_swap(&pq->data[idx], &pq->data[parent]);
    idx = parent;
}

return 1;
}

static int pq_empty(const PriorityQueue *pq) {
    return pq->size == 0;
}

static PQItem pq_pop(PriorityQueue *pq) {
    PQItem ret = pq->data[0];
    pq->data[0] = pq->data[--pq->size];

    int idx = 0;

    while (1) {
        int left = idx * 2 + 1;
        int right = idx * 2 + 2;
    }
}

```

```

    int smallest = idx;

    if (left < pq->size && pq_less(pq->data[left], pq->data[smallest])) {
        smallest = left;
    }

    if (right < pq->size && pq_less(pq->data[right], pq->data[smallest])) {
        smallest = right;
    }

    if (smallest == idx) break;

    pq_swap(&pq->data[idx], &pq->data[smallest]);
    idx = smallest;
}

return ret;
}

static void intvec_init(IntVector *v) {
    v->size = 0;
    v->capacity = 128;
    v->data = (int *)malloc((size_t)v->capacity * sizeof(int));
}

static void intvec_free(IntVector *v) {
    free(v->data);
    v->data = NULL;
    v->size = 0;
    v->capacity = 0;
}

static int intvec_push(IntVector *v, int x) {
    if (v->size >= v->capacity) {
        int newCap = v->capacity * 2;
        int *newData = (int *)realloc(v->data, (size_t)newCap * sizeof(int));

        if (!newData) return 0;

        v->data = newData;
        v->capacity = newCap;
    }
}

```

```

        v->data[v->size++] = x;
        return 1;
    }

static long long priority_for(const Board *b,
                             Algorithm alg,
                             HeuristicType hType,
                             int r,
                             int c,
                             int nextDigit,
                             long long g) {
    long long h = heuristic_value(b, r, c, nextDigit, hType);

    if (alg == ALG_UCS) return g;
    if (alg == ALG_GBFS) return h;

    return g + h;
}

static void free_search_result(SearchResult *res) {
    free(res->parent);
    free(res->parentAction);
    free(res->dist);
    intvec_free(&res->expandedTrace);
    memset(res, 0, sizeof(SearchResult));
}

static SearchResult search_solution(const Board *b, Algorithm alg, HeuristicType hType) {
    SearchResult res;
    memset(&res, 0, sizeof(SearchResult));

    int totalStates = state_count(b);
    int tc = target_count(b);

    res.parent = (int *)malloc((size_t)totalStates * sizeof(int));
    res.parentAction = (char *)malloc((size_t)totalStates * sizeof(char));
    res.dist = (long long *)malloc((size_t)totalStates * sizeof(long long));
    intvec_init(&res.expandedTrace);

    if (!res.parent || !res.parentAction || !res.dist || !res.expandedTrace.data) {
        fprintf(stderr, "Alokasi memori search gagal.\n");
    }
}

```

```

    return res;
}

for (int i = 0; i < totalStates; i++) {
    res.parent[i] = -1;
    res.parentAction[i] = '\0';
    res.dist[i] = INF;
}

PriorityQueue pq;
pq_init(&pq);

int startKey = encode_key(b, b->startR, b->startC, 0);

res.dist[startKey] = 0;

long long startPriority = priority_for(b, alg, hType, b->startR, b->startC, 0, 0);
pq_push(&pq, startKey, startPriority, 0);

clock_t startClock = clock();

while (!pq_empty(&pq)) {
    PQItem item = pq_pop(&pq);

    if (item.g != res.dist[item.key]) {
        continue;
    }

    int r, c, nextDigit;
    decode_key(b, item.key, &r, &c, &nextDigit);

    res.expandedCount++;
    intvec_push(&res.expandedTrace, item.key);

    if (r == b->goalR && c == b->goalC && nextDigit == tc) {
        res.found = 1;
        res.finalKey = item.key;
        res.finalCost = item.g;
        break;
    }

    for (int dir = 0; dir < 4; dir++) {

```

```

MoveResult mv;

if (!slide_move(b, r, c, nextDigit, dir, &mv)) {
    continue;
}

int nextKey = encode_key(b, mv.r, mv.c, mv.nextDigit);
long long newG = item.g + mv.moveCost;

if (newG < res.dist[nextKey]) {
    res.dist[nextKey] = newG;
    res.parent[nextKey] = item.key;
    res.parentAction[nextKey] = ACT[dir];

    long long pri = priority_for(b, alg, hType, mv.r, mv.c, mv.nextDigit, newG);
    pq_push(&pq, nextKey, pri, newG);
}
}

clock_t endClock = clock();
res.execMs = ((double)(endClock - startClock) * 1000.0) / CLOCKS_PER_SEC;

pq_free(&pq);
return res;
}

static SolutionPath reconstruct_solution(const Board *b, const SearchResult *res) {
    SolutionPath path;

    path.keys = NULL;
    path.actions = NULL;
    path.moveCount = 0;

    if (!res->found) return path;

    int count = 0;
    int k = res->finalKey;

    while (res->parent[k] != -1) {
        count++;
        k = res->parent[k];
    }

```

```

    }

    path.moveCount = count;
    path.keys = (int *)malloc((size_t)(count + 1) * sizeof(int));
    path.actions = (char *)malloc((size_t)count + 1);

    if (!path.keys || !path.actions) {
        free(path.keys);
        free(path.actions);

        path.keys = NULL;
        path.actions = NULL;
        path.moveCount = 0;
        return path;
    }

    path.actions[count] = '\0';

    k = res->finalKey;

    for (int idx = count; idx >= 1; idx--) {
        path.keys[idx] = k;
        path.actions[idx - 1] = res->parentAction[k];
        k = res->parent[k];
    }

    path.keys[0] = k;

    (void)b;
    return path;
}

static void free_solution_path(SolutionPath *path) {
    free(path->keys);
    free(path->actions);

    path->keys = NULL;
    path->actions = NULL;
    path->moveCount = 0;
}

static char display_char_at(const Board *b, int stateKey, int r, int c) {

```

```

int pr, pc, nextDigit;
decode_key(b, stateKey, &pr, &pc, &nextDigit);

if (r == pr && c == pc) return 'Z';

char ch = b->grid[r][c];

if (ch == 'Z') return '*';

if (is_digit_tile(ch)) {
    int d = ch - '0';

    if (d < nextDigit) return '*';
}

return ch;
}

static void print_board_state(FILE *out, const Board *b, int stateKey) {
    for (int i = 0; i < b->n; i++) {
        for (int j = 0; j < b->m; j++) {
            fputc(display_char_at(b, stateKey, i, j), out);
        }
        fputc("\n", out);
    }
}

static void print_solution_steps(FILE *out, const Board *b, const SolutionPath *path) {
    fprintf(out, "\nInitial\n");
    print_board_state(out, b, path->keys[0]);

    for (int i = 1; i <= path->moveCount; i++) {
        fprintf(out, "\nStep %d : %c\n", i, path->actions[i - 1]);
        print_board_state(out, b, path->keys[i]);
    }
}

static void print_expanded_trace(FILE *out, const Board *b, const SearchResult *res) {
    fprintf(out, "\nTrace konfigurasi yang ditinjau:\n");

    for (int i = 0; i < res->expandedTrace.size; i++) {
        int r, c, nextDigit;

```

```

int key = res->expandedTrace.data[i];

decode_key(b, key, &r, &c, &nextDigit);

fprintf(out,
        "Iterasi %d: posisi=(%d,%d), nextDigit=%d, cost=%lld\n",
        i + 1,
        r,
        c,
        nextDigit,
        res->dist[key]);
    }
}

#ifdef _WIN32
static struct termios oldTermios;
static int rawEnabled = 0;

static void disable_raw_mode(void) {
    if (rawEnabled) {
        tcsetattr(STDIN_FILENO, TCSAFLUSH, &oldTermios);
        rawEnabled = 0;
    }
}

static int enable_raw_mode(void) {
    if (tcgetattr(STDIN_FILENO, &oldTermios) == -1) return 0;

    atexit(disable_raw_mode);

    struct termios raw = oldTermios;
    raw.c_lflag &= (tcflag_t)~(ECHO | ICANON);
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    if (tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw) == -1) return 0;

    rawEnabled = 1;
    return 1;
}

static int read_byte_timeout(char *c, int ms) {

```

```

    fd_set set;
    struct timeval tv;

    FD_ZERO(&set);
    FD_SET(STDIN_FILENO, &set);

    tv.tv_sec = ms / 1000;
    tv.tv_usec = (ms % 1000) * 1000;

    int rv = select(STDIN_FILENO + 1, &set, NULL, NULL, &tv);

    if (rv > 0) {
        return read(STDIN_FILENO, c, 1) == 1;
    }

    return 0;
}
#endif

static int playback_key(void) {
#ifdef _WIN32
    int ch = _getch();

    if (ch == 0 || ch == 224) {
        int code = _getch();

        if (code == 77) return 1;
        if (code == 75) return -1;

        return 99;
    }

    if (ch == 27) return 2;
    if (ch == 'q' || ch == 'Q') return 0;
    if (ch == 'n' || ch == 'N' || ch == 'd' || ch == 'D') return 1;
    if (ch == 'p' || ch == 'P' || ch == 'a' || ch == 'A') return -1;
    if (ch == 'j' || ch == 'J') return 2;

    return 99;
#else
    char ch;

```

```

    if (read(STDIN_FILENO, &ch, 1) != 1) return 99;

    if (ch == 27) {
        char a, b;

        if (read_byte_timeout(&a, 40)) {
            if (a == '[' && read_byte_timeout(&b, 40)) {
                if (b == 'C') return 1;
                if (b == 'D') return -1;
            }
        }

        return 2;
    }

    if (ch == 'q' || ch == 'Q') return 0;
    if (ch == 'n' || ch == 'N' || ch == 'd' || ch == 'D') return 1;
    if (ch == 'p' || ch == 'P' || ch == 'a' || ch == 'A') return -1;
    if (ch == 'j' || ch == 'J') return 2;

    return 99;
#endif
}

static void playback_solution(const Board *b, const SolutionPath *path) {
    if (!path->keys || path->moveCount < 0) return;

    int step = 0;

#ifdef _WIN32
    enable_raw_mode();
#endif

    while (1) {
        clear_screen();

        if (step == 0) {
            printf("Playback - Initial\n");
        } else {
            printf("Playback - Step %d/%d : %c\n",
                step,
                path->moveCount,

```

```

        path->actions[step - 1]);
    }

    print_board_state(stdout, b, path->keys[step]);

    printf("\nPanah kanan/n/d = maju | Panah kiri/p/a = mundur | ESC/j = lompat step | q =
keluar\n");
    fflush(stdout);

    int cmd = playback_key();

    if (cmd == 0) {
        break;
    }

    if (cmd == 1 && step < path->moveCount) {
        step++;
    } else if (cmd == -1 && step > 0) {
        step--;
    } else if (cmd == 2) {
#ifdef _WIN32
        disable_raw_mode();
#endif

        printf("\nMasukkan step tujuan (0-%d): ", path->moveCount);

        int target;

        if (scanf("%d", &target) == 1) {
            if (target < 0) target = 0;
            if (target > path->moveCount) target = path->moveCount;

            step = target;
        }

        int ch;
        while ((ch = getchar()) != '\n' && ch != EOF) {
        }

#ifdef _WIN32
        enable_raw_mode();
#endif

```

```

    }
}

#ifdef _WIN32
    disable_raw_mode();
#endif
}

static int save_solution_file(const char *filename,
                             const Board *b,
                             Algorithm alg,
                             HeuristicType hType,
                             const SearchResult *res,
                             const SolutionPath *path) {
    FILE *out = fopen(filename, "w");

    if (!out) return 0;

    fprintf(out, "Algoritma: %s\n", algorithm_name(alg));
    fprintf(out, "Heuristic: %s\n", heuristic_name(hType));

    if (res->found) {
        fprintf(out, "Solusi Yang Ditemukan: %s\n", path->actions);
        fprintf(out, "Cost dari Solusi: %lld\n", res->finalCost);
        fprintf(out, "Waktu eksekusi: %.3f ms\n", res->execMs);
        fprintf(out, "Banyak iterasi yang dilakukan: %lld iterasi\n", res->expandedCount);

        print_solution_steps(out, b, path);
    } else {
        fprintf(out, "Solusi tidak ditemukan.\n");
        fprintf(out, "Waktu eksekusi: %.3f ms\n", res->execMs);
        fprintf(out, "Banyak iterasi yang dilakukan: %lld iterasi\n", res->expandedCount);
    }

    print_expanded_trace(out, b, res);

    fclose(out);
    return 1;
}

static void print_heuristic_info(void) {
    printf("\nPilihan heuristic:\n");

```

```

    printf("H1 = Manhattan distance ke target berikutnya.\n");
    printf("H2 = H1 dikali cost tile minimum.\n");
    printf("H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost
minimum.\n");
}

int main(void) {
    char inputPath[MAX_PATH_LEN];
    char algStr[32];
    char hStr[32];

    Algorithm alg;
    HeuristicType hType = HEUR_H1;

    Board board;
    char err[512];

    printf(">> Masukan file input:\n  ");

    if (scanf("%255s", inputPath) != 1) {
        printf("Input path tidak valid.\n");
        return 1;
    }

    if (!load_board(inputPath, &board, err, sizeof(err))) {
        printf("Input tidak valid: %s\n", err);
        return 1;
    }

    printf("\nPapan awal:\n");
    print_board_state(stdout, &board, encode_key(&board, board.startR, board.startC, 0));

    printf("\n>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)\n  ");

    if (scanf("%31s", algStr) != 1 || !parse_algorithm(algStr, &alg)) {
        printf("Algoritma tidak valid. Gunakan UCS, GBFS, atau A*.\n");
        free_board(&board);
        return 1;
    }

    if (alg == ALG_GBFS || alg == ALG_ASTAR) {
        print_heuristic_info();
    }
}

```

```

printf(">> Heuristic apa yang anda pilih? (H1/H2/H3)\n ");

if (scanf("%31s", hStr) != 1 || !parse_heuristic(hStr, &hType)) {
    printf("Heuristic tidak valid. Gunakan H1, H2, atau H3.\n");
    free_board(&board);
    return 1;
}
} else {
    strcpy(hStr, "H1");
    hType = HEUR_H1;
}

SearchResult result = search_solution(&board, alg, hType);
SolutionPath path = reconstruct_solution(&board, &result);

if (result.found && path.keys) {
    printf("\nSolusi Yang Ditemukan : %s\n", path.actions);
    printf("Cost dari Solusi : %lld\n", result.finalCost);

    print_solution_steps(stdout, &board, &path);
} else {
    printf("\nSolusi tidak ditemukan.\n");
}

printf("\n>> Waktu eksekusi: %.3f ms\n", result.execMs);
printf(">> Banyak iterasi yang dilakukan: %lld iterasi\n", result.expandedCount);

if (result.found && path.keys) {
    char ans[32];

    printf(">> Apakah Anda ingin melakukan playback? (Ya/Tidak):\n ");

    if (scanf("%31s", ans) == 1 && yes_answer(ans)) {
        playback_solution(&board, &path);
    }
}

char saveAns[32];

printf("\n>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):\n ");

```

```
if (scanf("%31s", saveAns) == 1 && yes_answer(saveAns)) {
    char outName[MAX_PATH_LEN];
    char resolvedOutPath[MAX_PATH_LEN * 2];

    printf(">> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:\n  ");

    if (scanf("%255s", outName) == 1) {
        if (strcmp(outName, "-") == 0) {
            strcpy(outName, "solusi.txt");
        }

        resolve_output_path(outName, resolvedOutPath, sizeof(resolvedOutPath));

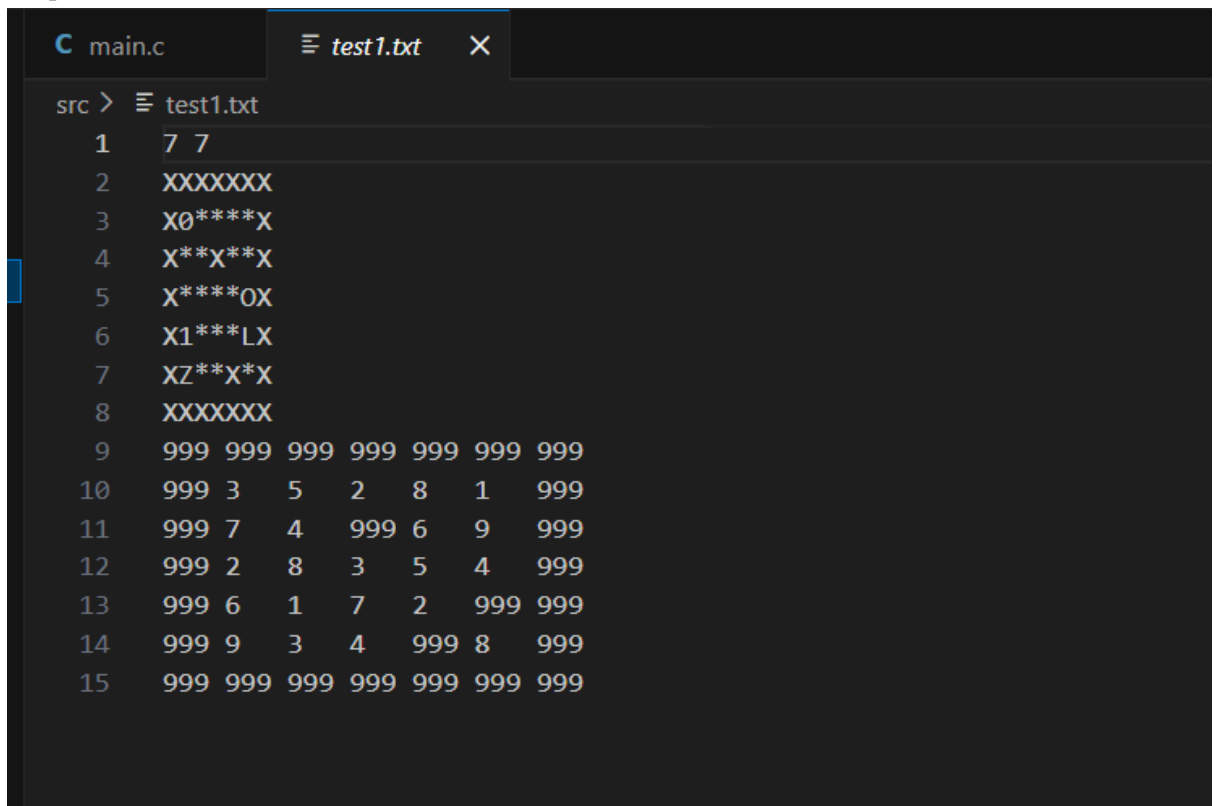
        if (save_solution_file(resolvedOutPath, &board, alg, hType, &result, &path)) {
            printf(">> Solusi disimpan pada %s\n", resolvedOutPath);
        } else {
            printf(">> Gagal menyimpan solusi ke %s\n", resolvedOutPath);
        }
    }
}

free_solution_path(&path);
free_search_result(&result);
free_board(&board);

return 0;
}
```

IV. Tangkapan Layar

1. Input:



```
src > test1.txt
1 7 7
2 XXXXXX
3 X0****X
4 X**X**X
5 X****OX
6 X1***LX
7 XZ**X*X
8 XXXXXX
9 999 999 999 999 999 999 999
10 999 3 5 2 8 1 999
11 999 7 4 999 6 9 999
12 999 2 8 3 5 4 999
13 999 6 1 7 2 999 999
14 999 9 3 4 999 8 999
15 999 999 999 999 999 999 999
```

Output:

UCS:

```
● PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> gcc -std=c11 -Wall -Wextra -O2 main.c -o ice_solver
❖ PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> .\ice_solver.exe
```

```
>> Masukan file input:
test1.txt
```

Papan awal:

```
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
```

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
UCS
```

Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial

```
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
```

Step 1 : R

```
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXX
```

Step 2 : U

```
XXXXXX
X0***X
X**X**X
X**Z*OX
X1***LX
```

Step 2 : U

```
XXXXXX
X0***X
X**X**X
X**Z*0X
X1***LX
X***X*X
XXXXXX
```

Step 3 : L

```
XXXXXX
X0***X
X**X**X
XZ***0X
X1***LX
X***X*X
XXXXXX
```

Step 4 : U

```
XXXXXX
XZ***X
X**X**X
X***0X
X1***LX
X***X*X
XXXXXX
```

Step 5 : D

```
XXXXXX
X*****X
X**X**X
X***0X
X***LX
XZ**X*X
XXXXXX
```

Step 6 : R

```
XXXXXX
X*****X
X**X**X
X***0X
X***LX
X**ZX*X
XXXXXX
```

Step 7 : U

```
Step 6 : R
XXXXXX
X****X
X**X**X
X***OX
X***LX
X**ZX*X
XXXXXX

Step 7 : U
XXXXXX
X****X
X**X**X
X**Z*OX
X***LX
X***X*X
XXXXXX

Step 8 : R
XXXXXX
X****X
X**X**X
X***ZX
X***LX
X***X*X
XXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 13 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
Ya
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ice_solver - src

Playback - Step 6/8 : R
XXXXXX
X****X
X**X**X
X***OX
X***LX
X**ZX*X
XXXXXX

Panah kanan/n/d = maju | Panah kiri/p/a = mundur | ESC/j = lompat step | q = keluar
```

```
EXPLORER PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - src

PENGUMPULAN
└─ src
   └─ ice_solver.exe
      └─ main.c
         └─ test1.txt
            └─ test2.txt
               └─ test3.txt
                  └─ test4.txt
                     └─ test5.txt
                        └─ test6.txt
                           └─ test
                              └─ jBUCTest1
                                 └─ README.md

Playback - Step 6/8 : R
XXXXXX
X****X
X**X**X
X***OX
X***LX
X**ZX*X
XXXXXX

Panah kanan/n/d = maju | Panah kiri/p/a = mundur | ESC/j = lompat step | q = keluar

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
jBUCTest1
>> Solusi disimpan pada ../test/jBUCTest1
PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src>
```

GBFS:

H1:

```
Tidak
❖ PS D:\1Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> .\ice_solver.exe
>> Masukkan file input:
test1.txt

Papan awal:
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
GBFS

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX

Step 1 : R
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXX
```

```
XXXXXXXX
X*****X
X**X**X
X***OX
X***LX
XZ**X*X
XXXXXX

Step 6 : R
XXXXXXXX
X*****X
X**X**X
X***OX
X***LX
X**Z*X*X
XXXXXX

Step 7 : U
XXXXXXXX
X*****X
X**X**X
X**Z*OX
X***LX
X**X*X*X
XXXXXX

Step 8 : R
XXXXXXXX
X*****X
X**X**X
X***ZX
X***LX
X**X*X*X
XXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 11 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbGBFStest1
>> Solusi disimpan pada ../test/JbGBFStest1
PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src>
```

H2:

```
PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> .\ice_solver.exe
>> Masukan file input:
test1.txt
```

Papan awal:

```
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
```

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
GBFS
```

Pilihan heuristic:

H1 = Manhattan distance ke target berikutnya.

H2 = H1 dikali cost tile minimum.

H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.

```
>> Heuristic apa yang anda pilih? (H1/H2/H3)
```

H2

Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial

```
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX
```

Step 1 : R

```
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXX
```


H3:

```
JbGBFSH2test1
>> Solusi disimpan pada ../test/JbGBFSH2test1
PS D:\1Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> .\ice_solver.exe
>> Masukkan file input:
    test1.txt

Papan awal:
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
    GBFS

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
    H3

Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXX

Step 1 : R
XXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXX
```

```

XXXXXXXX
X*****X
X**X**X
X***OX
X***LX
XZ**X*X
XXXXXXXX

Step 6 : R
XXXXXXXX
X*****X
X**X**X
X***OX
X***LX
X**ZX*X
XXXXXXXX

Step 7 : U
XXXXXXXX
X*****X
X**X**X
X**Z*OX
X***LX
X***X*X
XXXXXXXX

Step 8 : R
XXXXXXXX
X*****X
X**X**X
X***ZX
X***LX
X***X*X
XXXXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 9 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbGBFSH3test1
>> Solusi disimpan pada ../test/JbGBFSH3test1
PS D:\1Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src>

```

In 1.0

A*:

H1:

```
>> Masukan file input:
    test1.txt

Papan awal:
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
    A*

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
    H1

Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
XZ**X*X
XXXXXXX

Step 1 : R
XXXXXXX
X0***X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXXX
```

```
test
├── JbAH1test1
├── JbGBFSH1test1
├── JbGBFSH2test1
├── JbGBFSH3test1
├── JbUCStest1
└── README.md

Step 7 : U
XXXXXX
X****X
X***Z*X
XXXXXX

Step 8 : R
XXXXXX
X****X
X***Z*X
XXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 13 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbAH1test1
>> Solusi disimpan pada ../test/JbAH1test1
PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src>
```

H2:

```
>> Masukan file input:  
test1.txt
```

Papan awal:

```
XXXXXXX  
X0****X  
X**X**X  
X****OX  
X1***LX  
XZ**X*X  
XXXXXXX
```

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)  
A*
```

Pilihan heuristic:

H1 = Manhattan distance ke target berikutnya.

H2 = H1 dikali cost tile minimum.

H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.

```
>> Heuristic apa yang anda pilih? (H1/H2/H3)
```

H2

Solusi Yang Ditemukan : RULUDRUR

Cost dari Solusi : 87

Initial

```
XXXXXXX  
X0****X  
X**X**X  
X****OX  
X1***LX  
XZ**X*X  
XXXXXXX
```

Step 1 : R

```
XXXXXXX  
X0****X  
X**X**X  
X****OX  
X1***LX  
X**ZX*X  
XXXXXXX
```

Step 2 : U

```
XXXXXXX  
X0****X
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - src + v [ ] [ ] ... [ ] [ ] x

XXXXXX
X*****X
X**X**X
X***OX
X***LX
XZ**X*X
XXXXXX

Step 6 : R
XXXXXX
X*****X
X**X**X
X***OX
X***LX
X**ZX*X
XXXXXX

Step 7 : U
XXXXXX
X*****X
X**X**X
X**Z*OX
X***LX
X***X*X
XXXXXX

Step 8 : R
XXXXXX
X*****X
X**X**X
X***ZX
X***LX
X***X*X
XXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 13 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbAH2test1
>> Solusi disimpan pada ../test/JbAH2test1
PS D:\1Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> |
```

H3:

```
>> Masukan file input:
    test1.txt

Papan awal:
XXXXXX
X0****X
X**X**X
X***OX
X1***LX
XZ**Y*X
XXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
    A*

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
    H3

Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87

Initial
XXXXXX
X0****X
X**X**X
X***OX
X1***LX
XZ**Y*X
XXXXXX

Step 1 : R
XXXXXX
X0****X
X**X**X
X***OX
X1***LX
X**ZX*X
XXXXXX

Step 2 : U
XXXXXX
X0****X
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - src + v [ ] [ ] ... [ ] [ ] X

XXXXXX
X*****X
X**X**X
X***OX
X***LX
XZ**X*X
XXXXXX

Step 6 : R
XXXXXX
X*****X
X**X**X
X***OX
X***LX
X**ZX*X
XXXXXX

Step 7 : U
XXXXXX
X*****X
X**X**X
X**Z*OX
X***LX
X***X*X
XXXXXX

Step 8 : R
XXXXXX
X*****X
X**X**X
X***ZX
X***LX
X***X*X
XXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 13 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbAH3test1
>> Solusi disimpan pada ../test/JbAH3test1
PS D:\Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src>
```

2.Input:

```
main.c test2.txt
src > test2.txt
1 15 15
2 XXXXXXXXXXXXXXXXXXXX
3 X***X***X***X
4 X*****0X
5 X1*X***X***X
6 X*X*XX***X
7 X*X***5***X2X
8 X*****X***X
9 X*4*****XX*X
10 X*****X*****X
11 X*****0***XX
12 X*X*****3X
13 XX*****X
14 X***X*****X
15 X***Z*****X
16 XXXXXXXXXXXXXXXXXXXX
17 999 999 999 999 999 999 999 999 999 999 999 999 999 999
18 999 5 2 8 999 7 4 1 3 9 999 3 5 2 999
19 999 4 7 5 8 6 9 7 1 8 2 9 3 9 999
20 999 6 7 999 6 5 9 4 8 7 2 999 1 9 999
21 999 7 999 8 4 999 999 1 6 2 7 3 8 4 999
22 999 6 999 9 2 8 5 6 4 1 7 4 999 4 999
23 999 5 8 6 9 7 1 8 2 9 999 1 4 2 999
24 999 4 3 3 7 6 1 5 9 8 999 999 2 1 999
25 999 8 4 9 5 1 6 999 7 3 8 4 9 5 999
26 999 7 4 1 3 9 6 3 3 2 8 5 7 999 999
27 999 6 9 999 1 8 2 9 3 1 4 2 5 9 999
28 999 999 9 4 8 7 2 6 1 9 4 8 3 2 999
29 999 9 5 1 6 2 999 3 8 4 9 5 1 6 999
30 999 8 5 2 3 1 7 4 6 3 9 6 8 5 999
31 999 999 999 999 999 999 999 999 999 999 999 999 999
```

Output:

UCS:

```
PS D:\1Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> .\ice_solver.exe
>> Masukan file input:
test2.txt

Papan awal:
XXXXXXXXXXXXXXXXX
X***X***X***X
X*****0X
X1X***X***X
X*X**XX***X
X*X***5***X2X
X*****X***X
X*4*****X***X
X*****X***X
X*****0***XX
X**X*****3X
XX*****X
X*****X***X
X***_*****X
XXXXXXXXXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
UCS

Solusi Yang Ditemukan : URLDURDRDLDRULULDRUDLURURDRDL
Cost dari Solusi : 847

Initial
XXXXXXXXXXXXXXXXX
X***X***X***X
X*****0X
X1X***X***X
X*X**XX***X
X*X***5***X2X
X*****X***X
X*4*****X***X
X*****X***X
X*****0***XX
X**X*****3X
XX*****X
X*****X***X
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

ice_solver - src ⚠ + ▾ □ 🗑 ... | 🔍 ✕

```
X*****X***X
X*4*****X**X
X*****XZ*****X
X*****0*****XX
X**X*****3X
XX*****X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXX

Step 11 : D
XXXXXXXXXXXXXXXXX
X***y*****y***X
X*****X*****X
X**X*****X**X
X*X**XX*****X
X*X****5****X*X
X*****X***X
X*4*****XX**X
X*****y*****X
X*****0*****XX
X**X*****3X
XX*****X
X*****X*****X
X*****Z*****X
XXXXXXXXXXXXXXXXX

Step 12 : R
XXXXXXXXXXXXXXXXX
X***y*****X***X
X*****X*****X
X**X*****X**X
X*X**XX*****X
X*X****5****X*X
X*****X***X
X*4*****XX**X
X*****X*****X
X*****0*****XX
X**X*****3X
XX*****X
X*****X*****X
X*****Z*****X
XXXXXXXXXXXXXXXXX

Step 13 : U
XXXXXXXXXXXXXXXXX
```

In 1231. Col

EXPLORER

PENGUMPULAN

src

ice_solver.exe

main.c

test1.txt

test2.txt

test3.txt

test4.txt

test5.txt

test

JbAH1test1

JbAH2test1

JbAH3test1

JbGBFSH1test1

JbGBFSH2test1

JbGBFSH3test1

JbUCStest1

JbUCStest2

README.md

OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

powershell - src

+ v

Step 29 : L

XXXXXXXXXXXXXXXXX

X***X*****X***X

X*****X

X**X*****X**X

X*X**XX*****X

X*X*****X*X

X*****X***X

X*****XX**X

X*****XZ*****X

X*****O*****X

X**X*****X

XX*****X

X*****X*****X

X*****X

XXXXXXXXXXXXXXXXX

Step 30 : D

XXXXXXXXXXXXXXXXX

X***X*****X***X

X*****X

X**X*****X**X

X*X**XX*****X

X*X*****X*X

X*****X***X

X*****XX**X

X*****X*****X

X*****Z*****X

X**X*****X

XX*****X

X*****X*****X

X*****X

XXXXXXXXXXXXXXXXX

>> Waktu eksekusi: 0.000 ms

>> Banyak iterasi yang dilakukan: 255 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak):

Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):

Ya

>> Masukkan nama file output, atau '.' untuk solusi.txt di folder test:

JbUCStest2

>> Solusi disimpan pada ../test/JbUCStest2

GBFS:

```
>> Masukkan file input:  
test2.txt
```

Papan awal:

```
XXXXXXXXXXXXXXXXX  
X***X***X***X  
X*****0X  
X1*X***X***X  
X*X**XX***X  
X*X***5***X2X  
X*****Y***X  
X*4*****XX*X  
X*****X***X  
X*****0***XX  
X*X*****3X  
XX*****X  
X*****X***X  
X***7***X  
XXXXXXXXXXXXXXXXX
```

```
>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)  
GBFS
```

Pilihan heuristic:

H1 = Manhattan distance ke target berikutnya.

H2 = H1 dikali cost tile minimum.

H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.

```
>> Heuristic apa yang anda pilih? (H1/H2/H3)
```

H2

Solusi Yang Ditemukan : URLDURDRDLDRULDRURULDRDLDR

Cost dari Solusi : 874

Initial

```
XXXXXXXXXXXXXXXXX  
X***X***X***X  
X*****0X  
X1*X***X***X  
X*X**XX***X  
X*X***5***X2X  
X*****Y***X  
X*4*****XX*X  
X*****X***X  
X*****0***XX  
X*X*****3X  
XX*****X
```

```
XXXXXXXXXXXXXXXXX
X***X***X***X
X*****Z
X1***X***X
X*X*XX***X
X*X***5***X2X
X*****X***X
X*4*****XX***X
X*****X*****X
X*****0***XX
X*X*****3X
XX*****X
X***X***X***X
X*****X
XXXXXXXXXXXXXXXXX
```

Step 3 : L

```
XXXXXXXXXXXXXXXXX
X***X***X***X
XZ*****X
X1***X***X
X*X*XX***X
X*X***5***X2X
X*****X***X
X*4*****XX***X
X*****X*****X
X*****0***XX
X*X*****3X
XX*****X
X***X***X***X
X*****X
XXXXXXXXXXXXXXXXX
```

Step 4 : D

```
XXXXXXXXXXXXXXXXX
X***X***X***X
X*****X
X*X***X***X
X*X*XX***X
X*X***5***X2X
X*****X***X
X*4*****XX***X
X*****X*****X
X*****0***XX
XZ*X*****3X
XX*****X
```

EXPLORER

PENGUMPULAN

src

ice_solver.exe

main.c

test1.txt

test2.txt

test3.txt

test4.txt

test5.txt

test

JbAH1test1

JbAH2test1

JbAH3test1

JbGBFSH1test1

JbGBFSH2test1

JbGBFSH3test1

JbGBFStest2

JbUCStest1

JbUCStest2

README.md

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

powerShell - src

+ v

...

Step 27 : D

XXXXXXXXXXXXXXXXXX

X***X*****X***X

X*****X

X**X*****X**X

X*X**X*****X

X*X*****X*X

X*****X***X

X*****X**X

X*****X*****X

X**Z***0***X

X**X*****X

XX*****X

X*****X*****X

XXXXXXXXXXXXXXXXXX

Step 28 : R

XXXXXXXXXXXXXXXXXX

X***X*****X***X

X*****X

X**X*****X**X

X*X**X*****X

X*X*****X*X

X*****X***X

X*****X**X

X*****X*****X

X*****X***X

X**X*****X

XX*****X

X*****X*****X

X*****X*****X

X*****X*****X

XXXXXXXXXXXXXXXXXX

>> waktu eksekusi: 0.000 ms

>> Banyak iterasi yang dilakukan: 87 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak):

Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):

Ya

>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:

JbGBFStest2

>> Solusi disimpan pada /test/JbGBFStest2

> OUTLINE

A*:

```
>> Solusi disimpan pada ../test/JbGBFStest2
PS D:\1Kuliah\111 ITB Belajar\Semester 4\Stima\Tucil All\Tucil 3\Pengumpulan\src> .\ice_solver.exe
>> Masukkan file input:
test2.txt

Papan awal:
XXXXXXXXXXXXXXXXX
X***X***X***X
X*****0X
X1X***X***X
X*X**XX***X
X*X***5***X2X
X*****X***X
X*4*****XX***X
X*****X***X
X*****0***XX
X**X*****3X
XX*****X
X***X***X
X**2*****X
XXXXXXXXXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
A*

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H3

Solusi Yang Ditemukan : URLDURDRDLDRULDRUDLURURDRDL
Cost dari Solusi : 847

Initial
XXXXXXXXXXXXXXXXX
X***X***X***X
X*****0X
X1X***X***X
X*X**XX***X
X*X***5***X2X
X*****X***X
X*4*****XX***X
X***X***X
X*****0***XX
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

powershell - src

```
X*4*****XX**X
X*****Y*****X
X*****0*****XX
X**X*****3X
XX*****X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXX
```

Step 4 : D

```
XXXXXXXXXXXXXXXXX
X**X*****X***X
X*****X*****X
X*X*****X**X
X*X**XX*****X
X*X***5*****X2X
X*****X*****X
X*4*****XX**X
X*****X*****X
X*****0*****XX
XZ*X*****3X
XX*****X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXX
```

Step 5 : U

```
XXXXXXXXXXXXXXXXX
XZ*X*****X***X
X*****X*****X
X*X*****X**X
X*X**XX*****X
X*X***5*****X2X
X*****X*****X
X*4*****XX**X
X*****X*****X
X*****0*****XX
X*X*****3X
XX*****X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXX
```

Step 6 : R

```
XXXXXXXXXXXXXXXXX
X*ZX*****X***X
```

EXPLORER

PENGUMPULAN

src

ice_solver.exe

main.c

test1.txt

test2.txt

test3.txt

test4.txt

test5.txt

test

JbAH1test1

JbAH2test1

JbAH3test1

JbAtest2

JbGBFSH1test1

JbGBFSH2test1

JbGBFSH3test1

JbGBFStest2

JbUCStest1

JbUCStest2

README.md

OUTLINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

powershell - src

+ - [] []

Step 29 : L

XXXXXXXXXXXXXXXXXXXX

X***X*****X***X

X*****X

X**X*****X**X

X*X**X*****X

X*X*****X*X

X*****X**X

X*****XX**X

X*****XZ*****X

X*****O*****X

X**Y*****X

XX*****X

X*****X*****X

X*****X

XXXXXXXXXXXXXXXXXXXX

Step 30 : D

XXXXXXXXXXXXXXXXXXXX

X***X*****X***X

X*****X

X**X*****X**X

X*X**X*****X

X*X*****X*X

X*****X**X

X*****XX**X

X*****X*****X

X*****Z*****X

X**X*****X

XX*****X

X*****X*****X

X*****X

XXXXXXXXXXXXXXXXXXXX

>> Waktu eksekusi: 0.000 ms

>> Banyak iterasi yang dilakukan: 248 iterasi

>> Apakah Anda ingin melakukan playback? (Ya/Tidak):

Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):

Ya

>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:

JbAtest2

>> Solusi disimpan pada ../test/JbAtest2

3.Input:

```
main.c test3.txt X
src > test3.txt
1 4 5
2 XXXXX
3 X0*OX
4 XZ**X
5 XXXXX
6 0 0 0 0 0
7 0 2 3 4 0
8 0 1 5 6 0
9 0 0 0 0 0
```

Output:

UCS:

```
src
├── ice_solver.exe
├── main.c
├── test1.txt
├── test2.txt
├── test3.txt
├── test4.txt
├── test5.txt
└── test
    ├── JbAH1test1
    ├── JbAH2test1
    ├── JbAH3test1
    ├── JbAtest2
    ├── JbGBFSH1test1
    ├── JbGBFSH2test1
    ├── JbGBFSH3test1
    ├── JbGBFStest2
    ├── JbUCStest1
    ├── JbUCStest2
    ├── JbUCStest3
    └── README.md

>> Masukkan file input:
test3.txt

Papan awal:
XXXXX
X0*OX
XZ**X
XXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
UCS

Solusi Yang Ditemukan : UR
Cost dari Solusi : 9

Initial
XXXXX
X0*OX
XZ**X
XXXXX

Step 1 : U
XXXXX
XZ*OX
X***X
XXXXX

Step 2 : R
XXXXX
X**ZX
X***X
XXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 4 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
JbUCStest3
>> Solusi disimpan pada ../test/JbUCStest3
```

GBFS:

```
>> Masukan file input:
test3.txt

Papan awal:
XXXXX
X0*OX
XZ**X
XXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
GBFS

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi Yang Ditemukan : UR
Cost dari Solusi : 9

Initial
XXXXX
X0*OX
XZ**X
XXXXX

Step 1 : U
XXXXX
XZ*OX
X***X
XXXXX

Step 2 : R
XXXXX
X**ZX
X***X
XXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 3 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
JbGBFStest3
>> Solusi disimpan pada ../test/JbGBFStest3
```

A*:

```
test2.txt
test3.txt
test4.txt
test5.txt
test
  JbAH1test1
  JbAH2test1
  JbAH3test1
  JbAtest2
  JbAtest3
  JbGBFSH1test1
  JbGBFSH2test1
  JbGBFSH3test1
  JbGBFStest2
  JbGBFStest3
  JbUCStest1
  JbUCStest2
  JbUCStest3
① README.md

>> Masukan file input:
test3.txt

Papan awal:
XXXXX
X0*OX
XZ**X
XXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
A*

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H3

Solusi Yang Ditemukan : UR
Cost dari Solusi : 9

Initial
XXXXX
X0*OX
XZ**X
XXXXX

Step 1 : U
XXXXX
XZ*OX
X***X
XXXXX

Step 2 : R
XXXXX
X**ZX
X***X
XXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 4 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
JbAtest3
>> Solusi disimpan pada ../test/JbAtest3
```

4.Input:

```
main.c test4.txt x
src > test4.txt
1 15 15
2 XXXXXXXXXXXXXXXX
3 XZ0*****1***X
4 XXXXXXXXXXXXXXXX
5 X***3*****2**X
6 X*XXXXXXXXXXXXX
7 X*4*****5***X
8 XXXXXXXXXXXXXXXX
9 X***7*****6***X
10 X*XXXXXXXXXXXXX
11 X*****8*****X
12 XXXXXXXXXXXXXXXX
13 X*0*****9*****X
14 XXXXXXXXXXXXXXXX
15 XXXXXXXXXXXXXXXX
16 XXXXXXXXXXXXXXXX
17 999 999 999 999 999 999 999 999 999 999 999 999 999 999
18 999 1 1 1 1 1 1 1 1 1 1 1 1 999
19 999 999 999 999 999 999 999 999 999 999 999 999 1 999
20 999 1 1 1 1 1 1 1 1 1 1 1 1 999
21 999 1 999 999 999 999 999 999 999 999 999 999 999 999
22 999 1 1 1 1 1 1 1 1 1 1 1 1 999
23 999 999 999 999 999 999 999 999 999 999 999 999 1 999
24 999 1 1 1 1 1 1 1 1 1 1 1 1 999
25 999 1 999 999 999 999 999 999 999 999 999 999 999 999
26 999 1 1 1 1 1 1 1 1 1 1 1 1 999
27 999 999 999 999 999 999 999 999 999 999 999 999 1 999
28 999 1 1 1 1 1 1 1 1 1 1 1 1 999
29 999 999 999 999 999 999 999 999 999 999 999 999 999 999
30 999 999 999 999 999 999 999 999 999 999 999 999 999 999
31 999 999 999 999 999 999 999 999 999 999 999 999 999 999
```

Output:

UCS:

```
>> Masukan file input:
test4.txt

Papan awal:
XXXXXXXXXXXXXXXXX
XZ0*****1***X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXX*X
X***7***6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXX*X
X*0*****9***X
XXXXXXXXXXXXX
XXXXXXXXXXXXX
XXXXXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
UCS

Solusi Yang Ditemukan : RDLRDLRDL
Cost dari Solusi : 81

Initial
XXXXXXXXXXXXXXXXX
XZ0*****1***X
XXXXXXXXXXXXX*X
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXX*X
X***7***6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXX*X
X*0*****9***X
XXXXXXXXXXXXX
XXXXXXXXXXXXX
XXXXXXXXXXXXX

Step 1 : R
XXXXXXXXXXXXXXXXX
X*****Z
XXXXXXXXXXXXX*X
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXX*X
X***7***6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXX*X
X*0*****9***X
XXXXXXXXXXXXX
```

```
PENGUMPULAN
src
F ice_solver.exe
C main.c
F test1.txt
F test2.txt
F test3.txt
F test4.txt
F test5.txt
test
F JbAH1test1
F JbAH2test1
F JbAH3test1
F JbAtest2
F JbAtest3
F JbGBFSH1test1
F JbGBFSH2test1
F JbGBFSH3test1
F JbGBFSstest2
F JbGBFSstest3
F JbUCStest1
F JbUCStest2
F JbUCStest3
F JbUCStest4
README.md

X+XXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
X+XXXXXXXXXXXXX
X+*****ZX
XXXXXXXXXXXXXXX
X+0*****X
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX

Step 10 : D
XXXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
X+XXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
X+XXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****ZX
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX

Step 11 : L
XXXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
X+XXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+*****X
XXXXXXXXXXXXXXX
X+7*****X
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX
XXXXXXXXXXXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 25 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
JbUCStest4

>> Solusi disimpan pada ../test/JbUCStest4
```

GBFS:

```
>> Masukkan file input:
test4.txt

Papan awal:
XXXXXXXXXXXXXXXXX
XZ0*****1***X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXXXXXX
X***7*****6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXXXXXX
X*0*****9*****X
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
GBFS

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2

Solusi Yang Ditemukan : RDLDRDLDRDL
Cost dari Solusi : 81

Initial
XXXXXXXXXXXXXXXXX
XZ0*****1***X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXXXXXX
X***7*****6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXXXXXX
X*0*****9*****X
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX

Step 1 : R
XXXXXXXXXXXXXXXXX
X*****2X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
```

```
src
├── ice_solver.exe
├── main.c
├── test1.txt
├── test2.txt
├── test3.txt
├── test4.txt
├── test5.txt
└── test
    ├── JbAH1test1
    ├── JbAH2test1
    ├── JbAH3test1
    ├── JbAtest2
    ├── JbAtest3
    ├── JbGBFSH1test1
    ├── JbGBFSH2test1
    ├── JbGBFSH3test1
    ├── JbGBFSHtest2
    ├── JbGBFSHtest3
    ├── JbGBFSHtest4
    ├── JbUCStest1
    ├── JbUCStest2
    ├── JbUCStest3
    ├── JbUCStest4
    └── README.md

Step 10 : D
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*****ZX
XXXXXXXXXXXXXXXXX
X*0****9*****X
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX

Step 11 : L
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*****X
XXXXXXXXXXXXXXXXX
X*0****9*****ZX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 12 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbGBFSHtest4
>> Solusi disimpan pada ../test/JbGBFSHtest4
```

A*:

```
>> Masukan file input:
test4.txt

Papan awal:
XXXXXXXXXXXXXXXXX
XZ0*****1***X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXXXXXX
X***7*****6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXXXXXX
X*0*****9***X
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
A*

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H2

Solusi Yang Ditemukan : RDLDRDLDRDL
Cost dari Solusi : 81

Initial
XXXXXXXXXXXXXXXXX
XZ0*****1***X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
XXXXXXXXXXXXXXXXX
X***7*****6***X
X*XXXXXXXXXXXXX
X*****8*****X
XXXXXXXXXXXXXXXXX
X*0*****9***X
XXXXXXXXXXXXXXXXX
XXXXXXXXXXXXXXXXX

Step 1 : R
XXXXXXXXXXXXXXXXX
X*****7X
XXXXXXXXXXXXXXXXX
X***3*****2**X
X*XXXXXXXXXXXXX
X*4*****5***X
```

[illegible]

5.Input:

```
main.c test5.txt
src > test5.txt
1 10 10
2 XXXXXXXXXXXX
3 X*****X
4 X0*X*X**3X
5 X*****XX
6 XX*X*0Z**X
7 X****X***X
8 X*2**X*XXX
9 X*****X
10 X**X***1*X
11 XXXXXXXXXXXX
12 999 999 999 999 999 999 999 999 999 999
13 999 5 2 8 1 7 4 1 3 999
14 999 5 7 999 8 999 9 7 4 999
15 999 3 7 2 6 5 9 4 999 999
16 999 999 3 999 4 7 4 1 6 999
17 999 6 3 9 2 999 5 2 4 999
18 999 5 2 6 9 999 1 999 999 999
19 999 4 8 3 7 6 1 5 9 999
20 999 8 4 999 5 1 6 9 7 999
21 999 999 999 999 999 999 999 999 999 999
```

Output:

UCS:

```
>> Masukan file input:
test5.txt

Papan awal:
XXXXXXXXXX
X*****X
X0*X*X**3X
X*****XX
XX*X*0Z**X
X***X***X
X*2**X*XXX
X*****X
X**X***1*X
XXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
UCS
```

```
Solusi Yang Ditemukan : LULDRDLRULDRULDRUL
Cost dari Solusi : 326
```

```
Initial
XXXXXXXXXX
X*****X
X0*X*X**3X
X*****XX
XX*X*0Z**X
X***X***X
X*2**X*XXX
X*****X
X**X***1*X
XXXXXXXXXX
```

```
Step 1 : L
XXXXXXXXXX
X*****X
X0*X*X**3X
X*****XX
XX*X*0Z**X
X***X***X
X*2**X*XXX
X*****X
X**X***1*X
XXXXXXXXXX
```

```
Step 2 : U
XXXXXXXXXX
X***X***X
X0*X*X**3X
X*****XX
XX*X*0Z**X
X***X***X
X*2**X*XXX
X*****X
```

```
PENGUMPULAN
✓ src
  ▫ ice_solver.exe
  C main.c
  ▫ test1.txt
  ▫ test2.txt
  ▫ test3.txt
  ▫ test4.txt
  ▫ test5.txt
  ✓ test
    ▫ JbAH1test1
    ▫ JbAH2test1
    ▫ JbAH3test1
    ▫ JbAtest2
    ▫ JbAtest3
    ▫ JbAtest4
    ▫ JbGBFSH1test1
    ▫ JbGBFSH2test1
    ▫ JbGBFSH3test1
    ▫ JbGBFSH4test1
    ▫ JbUCStest1
    ▫ JbUCStest2
    ▫ JbUCStest3
    ▫ JbUCStest4
    ▫ JbUCStest5
  D README.md
  X*****X
  X**X*X***X
  X*****XX
  XX*X*0***X
  X***X*Z**X
  X***X*XXX
  X*****X
  X**X***X
  XXXXXXXXXX

  Step 22 : R
  XXXXXXXXXX
  X*****X
  X**X*X***X
  X*****XX
  XX*X*0***X
  X***X*Z**X
  X***X*XXX
  X*****X
  X**X***X
  XXXXXXXXXX

  Step 23 : U
  XXXXXXXXXX
  X*****X
  X**X*X***X
  X*****XX
  XX*X*0***X
  X***X*Z**X
  X***X*XXX
  X*****X
  X**X***X
  XXXXXXXXXX

  Step 24 : L
  XXXXXXXXXX
  X*****X
  X**X*X***X
  X*****XX
  XX*X*0***X
  X***X*Z**X
  X***X*XXX
  X*****X
  X**X***X
  XXXXXXXXXX

  >> Waktu eksekusi: 0.000 ms
  >> Banyak iterasi yang dilakukan: 86 iterasi
  >> Apakah Anda ingin melakukan playback? (Ya/Tidak):
  Tidak

  >> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
  Ya
  >> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
  JbUCStest5

  >> Solusi disimpan pada ../test/JbUCStest5
```

GBFS:

```
>> Masukan file input:
test5.txt

Papan awal:
XXXXXXXXXX
X*****X
X0*X*X*3X
X*****X
X(X*X*OZ**X
X***X***X
X*2**X)XX
X*****X
X**X***1*X
XXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
GBFS

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi Yang Ditemukan : LULDRDLRULDRURDULDRUL
Cost dari Solusi : 326

Initial
XXXXXXXXXX
X*****X
X0*X*X*3X
X*****X
X(X*X*OZ**X
X***X***X
X*2**X)XX
X*****X
X**X***1*X
XXXXXXXXXX

Step 1 : L
XXXXXXXXXX
X*****X
X0*X*X*3X
X*****X
X(X*XZO***X
X***X***X
X*2**X)XX
X*****X
X**X***1*X
XXXXXXXXXX

Step 2 : U
XXXXXXXXXX
```

PENGUMPULAN src ice_solver.exe main.c test1.txt test2.txt test3.txt test4.txt test5.txt test JbAH1test1 JbAH2test1 JbAH3test1 JbAtest2 JbAtest3 JbAtest4 JbGBFSH1test1 JbGBFSH2test1 JbGBFSH3test1 JbGBFSstest2 JbGBFSstest3 JbGBFSstest4 JbGBFSstest5 JbUCStest1 JbUCStest2 JbUCStest3 JbUCStest4 JbUCStest5 README.md OUTLINE	X*****X X**X*X***X X*****XX XX*X*O***X X***X*Z**X X***X*XXX X*****X X**X*****X XXXXXXXXXX Step 22 : R XXXXXXXXXX X*****X X**X*X***X X*****XX XX*X*O***X X***X*Z**X X***X*XXX X*****X X**X*****X XXXXXXXXXX Step 23 : U XXXXXXXXXX X*****X X**X*X***X X*****XX XX*X*O***X X***X*Z**X X***X*XXX X*****X X**X*****X XXXXXXXXXX Step 24 : L XXXXXXXXXX X*****X X**X*X***X X*****XX XX*X*Z**X X***X*XXX X*****X X**X*****X XXXXXXXXXX >> Waktu eksekusi: 0.000 ms >> Banyak iterasi yang dilakukan: 51 iterasi >> Apakah Anda ingin melakukan playback? (Ya/Tidak): Tidak >> Apakah Anda ingin menyimpan solusi? (Ya/Tidak): Ya >> Masukkan nama file output, atau '-' untuk solusi.txt di folder test: JbGBFSstest5 >> Solusi disimpan pada ../test/JbGBFSstest5
--	--

A*:

```
>> Masukan file input:
test5.txt

Papan awal:
XXXXXXXXXX
X*****X
X0*X*X**3X
X*****XX
XX*X*0Z**X
X***X***X
X*2**X*XXX
X*****X
X**X***1*X
XXXXXXXXXX

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*)
A*

Pilihan heuristic:
H1 = Manhattan distance ke target berikutnya.
H2 = H1 dikali cost tile minimum.
H3 = Estimasi rantai: posisi sekarang -> digit tersisa -> tujuan, dikali cost minimum.
>> Heuristic apa yang anda pilih? (H1/H2/H3)
H1

Solusi Yang Ditemukan : LULDRDLDRULDRURDULDRUL
Cost dari Solusi : 326

Initial
XXXXXXXXXX
X*****X
X0*X*X**3X
X*****XX
XX*X*0Z**X
X***X***X
X*2**X*XXX
X*****X
X**X***1*X
XXXXXXXXXX

Step 1 : L
XXXXXXXXXX
X*****X
X0*X*X**3X
X*****XX
XX*XZ0***X
X***X***X
X*2**X*XXX
X*****X
X**X***1*X
XXXXXXXXXX

Step 2 : U
XXXXXXXXXX
```

```
PENGUMPULAN
  src
  ice_solver.exe
  main.c
  test1.txt
  test2.txt
  test3.txt
  test4.txt
  test5.txt
  test
    JbAH1test1
    JbAH2test1
    JbAH3test1
    JbAtest2
    JbAtest3
    JbAtest4
    JbAtest5
    JbGBFSH1test1
    JbGBFSH2test1
    JbGBFSH3test1
    JbGBFSHtest2
    JbGBFSHtest3
    JbGBFSHtest4
    JbGBFSHtest5
    JbUCStest1
    JbUCStest2
    JbUCStest3
    JbUCStest4
    JbUCStest5
  README.md

X*****X
X**X**X**X
X*****XX
XX**X**X**X
X*****X**X
X*****X**X
X*****X
X**X*****X
XXXXXXX
Step 22 : R
XXXXXXX
X*****X
X**X**X**X
X*****XX
XX**X**X**X
X*****X**X
X*****X**X
X*****X
X**X*****X
XXXXXXX
Step 23 : U
XXXXXXX
X*****X
X**X**X**X
X*****XX
XX**X**X**X
X*****X**X
X*****X**X
X*****X
X**X*****X
XXXXXXX
Step 24 : L
XXXXXXX
X*****X
X**X**X**X
X*****XX
XX**X**X**X
X*****X**X
X*****X**X
X*****X
X**X*****X
XXXXXXX

>> Waktu eksekusi: 0.000 ms
>> Banyak iterasi yang dilakukan: 85 iterasi
>> Apakah Anda ingin melakukan playback? (Ya/Tidak):
    Tidak

>> Apakah Anda ingin menyimpan solusi? (Ya/Tidak):
    Ya
>> Masukkan nama file output, atau '-' untuk solusi.txt di folder test:
    JbAtest5
>> Solusi disimpan pada ../test/JbAtest5
```

V. Analisis hasil percobaan algoritma pathfinding

Berdasarkan hasil percobaan, Uniform Cost Search cenderung menghasilkan solusi dengan cost paling kecil atau sama dengan algoritma lain. Hal ini sesuai dengan karakteristik UCS yang selalu memproses state dengan total cost terkecil dari posisi awal. Karena semua cost traversal bernilai tidak negatif, UCS dapat menemukan solusi optimal dari sisi total cost. Namun, kelemahan UCS terlihat pada jumlah iterasi yang umumnya lebih besar. Hal ini terjadi karena UCS tidak memiliki informasi perkiraan arah menuju target, sehingga pencarian dapat melebar ke banyak state sebelum akhirnya mencapai tujuan.

Greedy Best First Search cenderung lebih cepat dalam beberapa percobaan karena algoritma ini menggunakan heuristic untuk memilih state yang terlihat paling dekat dengan target. Dengan bantuan heuristic, GBFS dapat langsung mengarah ke digit berikutnya atau titik tujuan tanpa harus mengeksplorasi semua state berdasarkan cost terkecil. Akan tetapi, solusi yang dihasilkan GBFS tidak selalu memiliki cost minimum. Hal ini disebabkan karena GBFS hanya mempertimbangkan estimasi jarak menuju target, bukan total cost yang sudah dikeluarkan dari posisi awal. Pada papan dengan variasi cost tile yang besar, jalur yang terlihat dekat belum tentu merupakan jalur dengan cost paling murah.

A* menunjukkan hasil yang lebih seimbang dibandingkan UCS dan GBFS. A* menggunakan kombinasi antara cost aktual dari posisi awal dan nilai heuristic menuju target. Karena itu, A* tetap memperhatikan biaya perjalanan seperti UCS, tetapi pencariannya lebih terarah karena menggunakan heuristic seperti GBFS. Pada

beberapa data uji, A* dapat menghasilkan cost solusi yang sama dengan UCS, tetapi dengan jumlah iterasi yang lebih sedikit. Hal ini menunjukkan bahwa heuristic membantu A* mengurangi jumlah state yang perlu ditinjau.

Pengaruh heuristic juga terlihat pada hasil percobaan. H1 merupakan heuristic paling sederhana karena hanya menggunakan jarak Manhattan menuju target berikutnya. H2 memberikan estimasi yang lebih realistis karena jarak Manhattan dikalikan dengan cost minimum tile. Sementara itu, H3 biasanya memberikan arah pencarian yang lebih informatif karena memperhitungkan perjalanan menuju seluruh digit yang tersisa secara berurutan hingga akhirnya mencapai titik tujuan. Oleh karena itu, pada beberapa kasus, A* dengan H3 dapat meninjau lebih sedikit state dibandingkan A* dengan H1 atau H2.

Meskipun demikian, heuristic yang lebih kompleks tidak selalu menjamin waktu eksekusi paling kecil pada semua test case. Hal ini karena waktu eksekusi tidak hanya dipengaruhi oleh jumlah iterasi, tetapi juga oleh biaya perhitungan heuristic pada setiap state. H3, misalnya, dapat memberikan estimasi yang lebih informatif, tetapi membutuhkan perhitungan lebih panjang karena harus menghitung estimasi melalui seluruh digit yang tersisa. Oleh karena itu, pada papan kecil, perbedaan waktu antarheuristic bisa tidak terlalu terlihat. Sebaliknya, pada papan yang lebih besar atau memiliki banyak digit, perbedaan heuristic dapat lebih memengaruhi jumlah state yang ditinjau.

Dari hasil percobaan secara keseluruhan, UCS cocok digunakan ketika prioritas utama adalah mendapatkan solusi dengan cost minimum. Namun, UCS dapat menjadi kurang efisien pada papan yang besar karena jumlah state yang ditinjau bisa cukup banyak. GBFS cocok digunakan ketika ingin mencari solusi dengan cepat, tetapi tidak terlalu mementingkan cost minimum. A* menjadi pilihan yang paling seimbang karena tetap mempertimbangkan cost aktual sekaligus menggunakan heuristic untuk mengarahkan pencarian.

Dengan demikian, algoritma yang paling stabil untuk permasalahan Ice Sliding Puzzle adalah A*. Algoritma ini dapat menghasilkan solusi yang baik dari sisi cost, sekaligus mengurangi jumlah pencarian dibandingkan UCS. Namun, hasil akhir tetap bergantung pada kualitas heuristic yang digunakan dan karakteristik papan pada masing-masing test case.

VI. Pranala Repository

https://github.com/Defaro123/Tucil3_13524046-.git

VII. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Farrell Limjaya

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)		✓
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	